

POLYTECH INTL LOGO

AfDB LOGO

End-of-Studies Project

Submitted on 15 September 2026
in partial fulfillment of the requirements
for the Professional Master's degree in Data Science

Presented and completed by :

Adem MRABET

Design and Implementation of a Knowledge-Graph-Based Conversational Agent for Institutional Delegation of Authority

Application to the African Development Bank's Delegation of Authority Matrix

Prepared within :

Polytech Intl

JURY

Dr.		Professor at Polytech Intl	President
Dr.		Professor at Polytech Intl	Reviewer
Mr.	Yasser AHMAD	Professional Supervisor	Industrial Supervisor
Mr.	Surya JOYMUNGUL	Professional Supervisor	Industrial Supervisor
Dr.	Marouene CHAIEB	Professor at Polytech Intl	Academic Supervisor

Professional Supervisor

Mr. Yasser AHMAD

Signature

Academic Supervisor

Dr. Marouene CHAIEB

Signature

Professional Supervisor

Mr. Surya JOYMUNGUL

Signature

Dedication

I dedicate this work:

To my father,

who taught me what quiet determination looks like long before I understood the word. You believed in me on the days I couldn't manage it myself, and every value I brought to this work started with something you showed me first.

To my mother,

for a love that never once wavered, for sacrifices whose size I only understood years later, and for being the calm I could always come back to, no matter how heavy this project got.

To my brother,

who has been a steady presence my whole life, and who never had to be asked.

To my girlfriend,

thank you doesn't cover it. You were patient on the nights this project took more than I had planned to give. You believed in me when I doubted myself, and you stood beside me through the hardest stretches without ever making me feel like a burden. Your support carried more of this than you know.

To my friends,

Ayoub and Alaa, whom I met during this Master's and could not be more grateful to have found. To Fayrouz and Houde, for always being there. And to Brahim, my childhood friend, for a friendship that has already outlasted this degree and will outlast plenty more.

Adem

Acknowledgements

I would like to thank Polytech Intl for hosting this project and for the training that made it possible, along with the teaching staff whose courses over the past two years shaped the way I approached this work.

My sincere thanks go to my academic supervisor, Dr. Marouene Chaieb, for his guidance throughout this internship and for the academic rigour he insisted on at every stage. His remarks consistently pushed the work toward something more defensible than what I would have produced on my own.

I am equally grateful to my professional supervisors, Mr. Yasser Ahmad and Mr. Surya Joymungul, for their availability and for the candid feedback they gave throughout the project. They tested the system directly, reported exactly what broke, and pushed back whenever something looked plausible rather than correct. That did more for the final result than any polite approval could have.

I also thank the African Development Bank Group for providing the real problem this work is built around, and for the access that made it possible to treat it as a genuine engineering problem rather than an academic exercise.

Finally, I thank the members of the jury for the honour of evaluating this work and for the time and attention they will give it.

Contents

Dedication	ii
Acknowledgements	iv
List of Figures	ix
List of Tables	x
List of Acronyms	xi
General Introduction	2
1 Project Framework, State of the Art, and Needs Analysis	5
Introduction	5
1.1 General Framework of the Project	5
1.2 Host Organisation	6
1.2.1 Presentation of Polytech Intl	6
1.2.2 Project Context and the Client Organisation	6
1.3 Study of the Existing System and Critical Analysis	7
1.3.1 Context of Delegation of Authority in Development Banking	7
1.3.1.1 Definition and Stakes of the DAM	7
1.3.1.2 Governance and Compliance Requirements	8
1.3.1.3 Challenges in Consulting the DAM in Practice	8
1.3.2 Study of Existing Systems	8
1.3.2.1 Manual Consultation and Document Search	8
1.3.2.2 Generic Document Question-Answering Tools	9
1.3.2.3 Critical Analysis: Limitations of Current Approaches	9
1.4 Functional Needs Analysis	10
1.4.1 Functional Requirements	10
1.4.2 Non-Functional Requirements	12
1.4.3 Project Constraints	13
1.5 Proposed Solution	13
1.5.1 General Presentation of the Solution	13

1.5.2	Global System Architecture	14
1.5.3	Selected Technologies	15
	Conclusion	16
2	Data Understanding: Analysis of the DAM Source Document	17
	Introduction	17
2.1	Source Document Presentation	17
2.1.1	Source and Context	17
2.1.2	Hierarchical Structure of the Document	18
2.1.3	Table Layout and the Authority Notation	18
2.2	Exploratory Analysis of the Document	19
2.2.1	Global Overview and Node Distribution	19
2.2.2	Authority Code Distribution	19
2.2.3	Role and Actor Analysis	20
2.2.4	Threshold Variants and Monetary Bands	20
2.2.5	Cross-References and Redirects	20
2.2.6	Consolidated View	21
2.3	Extraction Quality Assessment	21
2.3.1	Text Layer Inspection	21
2.3.2	Rotated Column Headers	22
2.3.3	Fused Footnote Digits and Authority Codes	22
2.3.4	Cross-Row Text Bleeding and Title Corruption	22
2.3.5	Split Rows in the Non-Sovereign Operations Chapter	23
2.3.6	Consolidated Data Issues Summary	23
	Conclusion	24
3	Data Engineering Pipeline and Knowledge Graph Construction	25
	Introduction	25
3.1	Processing Environment	25
3.1.1	Motivation for a Deterministic, Rule-Based Pipeline	25
3.1.2	Why Not a Distributed Processing Framework	25
3.1.3	Tooling	26
3.2	Pipeline Design	26
3.2.1	Global Architecture	26
3.2.2	Data Flow and Modular Organisation	27
3.3	Data Ingestion	29
3.4	Document Processing and Normalisation	29
3.4.1	Row Reconstruction	29
3.4.2	Task Block Detection	29

3.4.3	Title and Reference Cleaning	29
3.4.4	Geometric Attribution of Authority Codes	30
3.4.5	Role Attribution	30
3.5	Node Construction and Validation	30
3.6	Knowledge Graph Modelling	31
3.6.1	Why a Graph	31
3.6.2	Node and Edge Schema	31
3.6.3	Graph Statistics	32
3.7	Pipeline Evaluation	32
3.7.1	Execution and Caching	32
3.7.2	Data Quality Outcomes	33
3.7.3	Observed Limitations	33
	Conclusion	34
4	Retrieval, Grounded Generation and Agent Design	35
	Introduction	35
4.1	Retrieval Methodology	35
4.1.1	Identifier Fast Path	35
4.1.2	Free-Text Resolution	36
4.1.3	Why Lexical Retrieval Rather Than Embeddings	36
4.1.4	Typo Tolerance	36
4.2	Evaluation Protocol and Confidence Semantics	37
4.3	Deterministic Answer Construction	38
4.3.1	Intent Detection	38
4.3.2	Mandatory Obligation Surfacing	38
4.3.3	Cross-Reference Following	38
4.3.4	Honest Refusal	38
4.4	Auxiliary Query Types	39
4.5	Hybrid Language Model Layer	39
4.6	Grounded Generation and the Verification Check	40
4.6.1	The Contract Given to the Model	40
4.6.2	The Verification Step	40
4.6.3	Answer Length and the Fallback Rate	42
4.7	Conversational Capabilities	42
4.7.1	Context Carry-Over	42
4.7.2	Tone Adaptation	42
4.7.3	Multilingual Questions	43
4.8	Worked Examples	43
	Conclusion	44

5	Implementation Results and Delivered Platform	45
	Introduction	45
5.1	The Delivered Application	45
5.1.1	Overview and Architecture	45
5.1.2	Authentication and Access Control	45
5.1.3	The Chat Interface	46
5.1.4	Voice Input	47
5.1.5	Document Attachment	48
5.1.6	Conversation Export	48
5.1.7	Conversation Sharing	49
5.1.8	Interface Localisation	49
5.2	Analytics and Forecasting	50
5.2.1	The Analytics Dashboard	50
5.2.2	Governance Indicators	51
5.2.3	Usage Indicators	52
5.2.4	Forecasting	53
5.2.4.1	Rationale	53
5.2.4.2	Method	54
5.2.4.3	The Data Requirement, Stated Plainly	54
5.2.4.4	Bottleneck Projection	54
5.2.4.5	What Is Deliberately Not Forecast	55
5.3	REST Interface	55
5.4	Testing and Validation	56
5.5	Deployment	58
	Conclusion	58
	General Conclusion	60
	Bibliography and Webography	63

List of Figures

1.1	Global architecture of the proposed system, from source document to user-facing answer.	14
3.1	The eight-stage data engineering pipeline, from raw PDF extraction to a validated node set.	28
3.2	Excerpt of the knowledge graph around activity 3.111, showing the containment edge from its parent process and the responsibility edges connecting it to its approving and informed roles.	32
4.1	The grounded generation flow. The model may only rephrase facts that have already been verified, and any rephrasing that drops or alters a fact is discarded in favour of the deterministic template answer.	41

List of Tables

1.1	Authority code notation used in the DAM.	7
1.2	Functional requirements.	10
1.3	Non-functional requirements.	12
1.4	Project constraints.	13
1.5	Selected technologies and their role in the system.	15
2.1	Distribution of node types across the processed subset.	19
2.2	Consolidated summary of extraction issues found in the source document.	23
3.1	Stages of the data engineering pipeline.	26
3.2	Data quality indicators after node construction.	31
3.3	Edge types in the knowledge graph.	32
4.1	Resolution methods and their confidence semantics.	37
4.2	Language model providers and operating modes.	39
5.1	Governance indicators derived from the knowledge graph.	51
5.2	Usage indicators derived from the query log.	52
5.3	REST API endpoints exposed by the backend.	55
5.4	Breakdown of the automated test suite.	56

List of Acronyms

Acronym	Meaning
AfDB	African Development Bank Group
API	Application Programming Interface
ARIMA	Autoregressive Integrated Moving Average
ASR	Automatic Speech Recognition
CRISP-DM	Cross-Industry Standard Process for Data Mining
DAM	Delegation of Authority Matrix
DDG	Deputy Director-General
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
MVP	Minimum Viable Product
NSO	Non-Sovereign Operations
OOM	Out Of Memory
PDF	Portable Document Format
QR	Quick Response (code)
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
RDG	Regional Director General
RDVP	Regional Development, Integration and Business Delivery Vice-Presidency
REST	Representational State Transfer
RTL	Right-To-Left (text direction)
SO	Sovereign Operations
TF-IDF	Term Frequency – Inverse Document Frequency
UI	User Interface
VP	Vice-President

General Introduction

Every large institution eventually writes down who is allowed to decide what. In a development bank, that question carries real weight: an approval given by the wrong person is not a procedural detail but a governance failure, and one that auditors will find. The African Development Bank Group records these rules in a document called the Delegation of Authority Matrix, usually shortened to DAM. It states, activity by activity, which role may initiate a task, which must check or verify it, which reviews it, which approves it, and which simply has to be told that it happened.

The document works. The problem is reaching it. In its complete form the DAM runs past two hundred and fifty pages, and almost all of that content sits inside wide tables whose column headers are printed sideways to fit the page. Authority is expressed through a compact code, a single letter sometimes carrying a superscript digit, and the meaning of that code changes depending on the digit attached to it. A staff member who needs to know whether they can sign off on a mission programme has to find the right row in the right table in the right chapter, read a rotated header to work out which column belongs to which role, and then decode the letter sitting at the intersection. People do this. They do it slowly, and they do not always do it correctly.

The obvious modern answer is to ask a chatbot. That answer fails in a way that matters here. Language models produce fluent text, and fluent text about who holds financial approval authority is dangerous when it is wrong. A model that invents a plausible-sounding role name, or quietly drops one of the three people who must be informed, produces an answer that looks exactly as trustworthy as a correct one. For a compliance document, an invented answer is worse than no answer at all.

This thesis describes a system built around that constraint rather than in spite of it. The DAM is parsed into a structured knowledge base, that knowledge base is turned into a graph of activities and responsibilities, and questions are answered by retrieving real records from the graph. A language model is present, but it is never allowed to decide what the facts are. It is given facts that have already been retrieved and asked only to phrase them naturally, and its output is checked against those facts, word for word, before a user ever sees it. If the model drops a role name or alters one, the rewritten answer is discarded and a plain template answer is shown instead.

Objectives

The work set out to deliver three things. The first was a faithful machine-readable representation of the DAM's content, one that could be checked against the printed document rather than trusted on the strength of the code that produced it. The second was a question-answering agent that resolves ordinary questions, typed the way people actually type them, to the correct activity and returns its recorded responsibilities. The third was a working, hosted application rather than a notebook: something a supervisor or an examiner can open in a browser and interrogate directly.

A fourth objective emerged during the work and is worth stating because it shaped everything else. The system had to be honest about its own limits. Asked about an activity that does not exist, it says so. Asked something outside the DAM's subject matter, it declines rather than guessing. This behaviour is not a fallback for failure cases; it is the property that makes the correct answers worth anything.

A second phase of the project extended the platform from a question-answering tool into something closer to an institutional application: authenticated access, voice input, document attachment, a fully translated interface, exportable and shareable conversations, and an analytics layer carrying governance indicators and a forecasting capability. Chapter 5 describes that phase in detail.

Scope

The complete DAM exceeds two hundred and fifty pages. Processing all of it was neither necessary nor realistic within the time available, so the work was scoped to a seventy-nine-page subset covering three chapters of the document, including the Non-Sovereign Operations chapter that was originally outside the intended scope and was added once its structure proved compatible with the parser. Everything reported in this thesis, including every count and every quality measure, refers to that subset. Nothing in the design prevents the remaining chapters from being processed by the same pipeline, and the reasons for believing this are set out in Chapter 3.

Structure of the Thesis

Chapter 1 places the project in context. It introduces the host institution and the client organisation, examines how the DAM is consulted today and where that process breaks down, reviews what existing tools offer and why they fall short for this document, and closes with the requirements and the architecture chosen in response.

Chapter 2 examines the source document itself. Rather than a dataset of rows and columns, the raw material here is a PDF whose structure is visual, so this chapter is an exploration of how that structure encodes meaning and of the specific ways automated extraction fails on it.

Chapter 3 describes the engineering pipeline that turns the document into data: the stages, the transformations, the validation applied at each step, and the knowledge graph built on top of the result.

Chapter 4 covers the agent itself, which is to say retrieval, answer construction, the language model layer, and the verification mechanism that keeps that layer honest.

Chapter 5 presents the delivered platform: the application and its interface, the access control and collaboration features, the analytics and forecasting layer, the testing regime, and how the whole thing is deployed. The general conclusion then reviews what was achieved, what was not, and what a continuation of this work would address first.

Chapter 1

Project Framework, State of the Art, and Needs Analysis

Introduction

This chapter establishes the setting of the project. It presents the organisation that hosted the work and the institution whose document motivated it, then looks closely at how that document is used in practice today. That examination matters more than it might appear: the weaknesses of the current process are what define the requirements, and several design decisions taken later in this thesis only make sense once those weaknesses are understood concretely. The chapter then surveys the tools that already exist for this class of problem, explains why none of them is sufficient here, and ends with the requirements, constraints, and proposed architecture that follow from all of it.

1.1 General Framework of the Project

This work was carried out as a final-year internship project within the Master's programme in Data Sciences at Polytech Intl, over a six-month internship period, of which a little more than four months went into active development of the system described here. The remaining time covered the initial framing of the subject, familiarisation with the source material, and the preparation of this report.

The subject sits at an intersection that is becoming common in applied data science. On one side there is a document processing problem: a PDF whose meaning is carried by layout rather than by markup, requiring geometric reasoning to recover. On the other there is a question-answering problem of the kind now usually associated with large language models. The interesting part of the project is the tension between them. Document extraction rewards determinism, since a rule that works can be verified against the page it was derived from. Language models offer fluency and flexibility but no guarantee of faithfulness. Building something useful meant using both without letting the second undermine the first.

1.2 Host Organisation

1.2.1 Presentation of Polytech Intl

Polytech Intl is a private higher education institution in Tunisia offering engineering and Master's level programmes in computing, data science, and related engineering disciplines. Its programmes combine academic coursework with project-based work and a mandatory final internship, on the principle that graduates should leave having built something under conditions resembling professional practice rather than only having studied how such things are built.

The Master's programme in Data Sciences covers statistics and machine learning, data engineering and distributed processing, and the software engineering practices needed to turn analysis into a deliverable system. The final project is expected to demonstrate competence across that full range rather than in one part of it, which is one reason this project was carried through to a deployed application instead of stopping at a validated dataset.

1.2.2 Project Context and the Client Organisation

Polytech Intl acted as the host institution for this internship, providing both the academic supervision and the working framework for the project. The subject matter, however, came from outside: the document at the centre of this work belongs to the African Development Bank Group, which acted as the client organisation for the project.

The African Development Bank Group is a multilateral development finance institution founded in 1964, headquartered in Abidjan, Côte d'Ivoire. It finances projects and programmes across its regional member countries in areas including energy, agriculture, transport infrastructure, water, education, and private sector development, and it operates through both sovereign lending to governments and non-sovereign operations with private entities. An institution of that size and mandate necessarily formalises its internal decision rights, and the Delegation of Authority Matrix is the instrument through which it does so.

Working with a document from a real institution changed the character of the project in a useful way. The material could not be cleaned up or simplified to suit the parser, the correctness bar was set by what the printed pages actually say rather than by a metric chosen for convenience, and every claim in this report about what the system extracts can be checked against a page of the source document.

1.3 Study of the Existing System and Critical Analysis

1.3.1 Context of Delegation of Authority in Development Banking

1.3.1.1 Definition and Stakes of the DAM

A delegation of authority framework answers a narrow question with wide consequences: for a given institutional act, who is permitted to perform which part of it. In the Bank's case the DAM expresses this as a matrix. Rows correspond to activities, organised hierarchically from broad chapters down through processes to individual tasks and, where a monetary threshold changes the answer, to variants of a task. Columns correspond to roles, from country office staff through sector managers and directors up to vice-presidents and the Board. The cell where an activity meets a role carries a code stating what that role does for that activity.

The codes are compact, and their compactness is part of the problem. Table 1.1 gives the notation.

Table 1.1: Authority code notation used in the DAM.

Code	Meaning
I	Initiate. The role starts the activity.
C1, C2	Check and verify. The role must confirm the activity before it proceeds.
C3, C4	Consult. The role must be consulted, typically the Board, the Board of Governors, or an independent function.
R	Review. The role reviews the activity.
A	Approve. The role holds final approval authority.
(i)	Informed. The role must be told about the activity but takes no action on it.

Two features of this notation deserve attention because they cause trouble later. First, the letter C covers two unrelated concepts separated only by the digit that follows it. C1 and C2 mean check and verify; C3 and C4 mean consult, in the governance sense of referring a matter to the Board or to an independent function. Treating both as a single "C" would merge two different obligations into one, and any system that maps the word "check" onto the bare letter C makes exactly that mistake. Second, the informed marker is not a letter at all but a parenthesised character, which behaves differently from the other codes at every stage of processing.

1.3.1.2 Governance and Compliance Requirements

The DAM is not advisory. It is the reference against which internal audit assesses whether a decision was taken by someone entitled to take it, and it exists partly to satisfy the governance expectations that come with public and multilateral funding. Three consequences follow for any system built on top of it.

The first is that a wrong answer has a cost beyond inconvenience. If a staff member approves something on the strength of an incorrect lookup, the error surfaces during audit, not during the work. The second is that completeness matters as much as correctness. A recorded check-and-verify obligation is mandatory, so an answer that names the approver but omits the role that must verify the task is incomplete in a way that can mislead, even though nothing in it is false. The third is traceability: an answer is only useful if it can be traced back to the row of the document it came from, because a user who cannot verify an answer has no basis for relying on it.

1.3.1.3 Challenges in Consulting the DAM in Practice

Reading the DAM is a mechanical exercise that the document's own layout makes harder than it needs to be. The tables are wide, so role headers are rotated ninety degrees to fit within the page margins. A reader must therefore map a sideways label at the top of a column to a code appearing many centimetres below it, and must do so correctly for the column immediately adjacent as well, since neighbouring roles frequently hold different authority over the same activity.

Superscript footnote digits sit directly against the authority codes, and distinguishing a footnote marker from part of the code requires knowing which is which. Some rows contain no responsibilities at all, holding instead a cross-reference to another activity elsewhere in the document, which means the reader must follow a pointer and start again in a different chapter. New staff, who are exactly the population most in need of the document, are also the least equipped to navigate it quickly.

The practical result is that the DAM is consulted less often than it should be. People ask a colleague instead, or they rely on what was done the last time a similar activity came up. Neither habit is auditable, and both propagate errors quietly.

1.3.2 Study of Existing Systems

1.3.2.1 Manual Consultation and Document Search

The existing process is manual consultation of the PDF, supported by whatever search the reader's PDF viewer provides. Full-text search finds character strings, which

helps when the user already knows the exact title of an activity or its identifier, and helps very little otherwise. Searching for the word “approve” returns hundreds of matches across the document, none of them ranked by relevance to the activity the user has in mind. Searching for a partially remembered activity name returns nothing at all unless the remembered words match the printed ones exactly.

More importantly, text search returns locations, not answers. It can tell a user that a term appears on page 24. It cannot tell them who approves the activity described there, because doing so would require understanding that the page contains a table, that the table has rotated headers, and that a particular letter in a particular cell means approval by a particular role.

1.3.2.2 Generic Document Question-Answering Tools

A second category of existing solution is the general-purpose document assistant: upload a PDF, ask questions in natural language, receive answers. These tools typically work by splitting the document into passages, embedding those passages as vectors, retrieving the passages most similar to the question, and passing them to a language model that writes an answer.

This approach performs well on prose. It performs poorly on this document for a reason that is structural rather than incidental. When a page of the DAM is converted to plain text, the spatial relationships that carry its meaning are destroyed. The rotated header that identified a column becomes a stray string somewhere in the extracted text, disconnected from the codes beneath it. A retrieved passage may therefore contain the activity name and the letter A without preserving the information that the A belonged to the column headed by a specific director. The model receives text that has already lost the answer and produces something plausible from what remains.

1.3.2.3 Critical Analysis: Limitations of Current Approaches

Neither existing approach fails for want of engineering polish. They fail because of assumptions that do not hold for this document.

Text search assumes the user can express their need in the document’s own vocabulary. In practice a user asks about “the quarterly mission programme” while the document may phrase it differently, and the search returns nothing useful.

Generic retrieval-augmented assistants assume that flattening a document into passages preserves its meaning. For a table whose semantics are positional, flattening discards precisely the information the question depends on.

Both approaches, finally, share a deeper problem: they have no notion of being wrong. A search returns results and a language model returns text, and in neither case does the system distinguish between an answer it can support and one it cannot. For a compliance reference, that distinction is the whole point.

1.4 Functional Needs Analysis

1.4.1 Functional Requirements

The functional requirements fall into two groups. Requirements FR1 to FR10 concern the core question-answering capability and were derived from the way the document is actually consulted today. Requirements FR11 to FR16 concern the platform capabilities added in the project's second phase, which extend the system from a lookup tool into an application usable inside an institution.

Table 1.2: Functional requirements.

Ref.	Requirement	Description
FR1	Direct identifier lookup	Given a valid activity identifier, the system returns that activity's recorded authority information with full confidence and no ambiguity.
FR2	Natural-language lookup	Given a question describing an activity by name rather than by identifier, the system resolves it to the correct activity and reports how confident that resolution is.
FR3	Authority-role queries	For a resolved activity, the system answers who approves, reviews, checks, initiates, or must be informed, using the document's own notation.
FR4	Mandatory obligation surfacing	Whatever the question asked, the system always surfaces a recorded check-and-verify obligation and, where present, the informed parties.
FR5	Cross-reference resolution	Where an activity's row is a pointer to another activity, the system follows the pointer automatically rather than reporting that no responsibilities exist.

Ref.	Requirement	Description
FR6	Typing and phrasing tolerance	The system tolerates common misspellings and phrasing variation without requiring the document's exact vocabulary.
FR7	Honest refusal	For an identifier that does not exist, or a question outside the document's subject matter, the system says so explicitly instead of returning a best guess.
FR8	Conversational continuity	A follow-up question that does not restate its subject is answered against the activity discussed immediately before.
FR9	Structural analytics	The system exposes an overview of the document's own structure: node counts, authority-code distribution, roles most frequently involved, and per-chapter breakdowns.
FR10	Multilingual questions	Questions asked in French, Spanish, Portuguese, or Arabic are answered in the language they were asked in, without translating role names or identifiers.
FR11	Authenticated access	Access to the application requires authentication, and analytical features are restricted to authorised roles.
FR12	Voice input	A user can dictate a question instead of typing it, in any of the supported interface languages.
FR13	Document attachment	A user can attach a supporting document to a conversation and ask questions about its content, with answers from that source clearly distinguished from answers grounded in the DAM.
FR14	Fully localised interface	Every element of the interface, not only the conversation, is available in each supported language, including right-to-left layout for Arabic.
FR15	Conversation export	A user can export a conversation as a PDF document preserving the questions, the answers, and the provenance metadata attached to each answer.

Ref.	Requirement	Description
FR16	Conversation sharing	A user can share a read-only view of a conversation with a colleague through a link encoded as a QR code.

1.4.2 Non-Functional Requirements

Table 1.3: Non-functional requirements.

Ref.	Requirement	Description
NFR1	Factual fidelity	No answer may contain a role, code, or qualification that is not recorded in the source document for that activity. This requirement takes precedence over fluency, speed, and every other consideration.
NFR2	Traceability	Every answer must be attributable to a specific activity record, so that a user can verify it against the printed document.
NFR3	Availability without external services	Core lookup must work when no language model provider is reachable, degrading to template answers rather than failing.
NFR4	Response time	Answers should return within a few seconds on the free hosting tier used for deployment, which rules out re-parsing the document per request.
NFR5	Reproducibility	The same question must produce the same underlying facts on every run, independent of any model's sampling behaviour.
NFR6	Portability	The system must run identically on a developer machine and on the hosting platform, without environment-specific configuration.
NFR7	Usability on any device	The interface must remain usable on a phone, since consultation often happens away from a desk.
NFR8	Credential protection	Authentication credentials must never be stored in recoverable form, and session tokens must expire.

Ref.	Requirement	Description
NFR9	Upload safety	Attached documents must be constrained by type and size, and must never be executed or interpreted as anything other than text to be read.

1.4.3 Project Constraints

Table 1.4: Project constraints.

Constraint	Consequence for the project
No labelled training data	No query-and-answer pairs and no annotated tables existed, which ruled out training a supervised model and pushed retrieval toward lexical methods that need no training corpus.
Document scope	The full DAM exceeds 250 pages. Work was scoped to a 79-page subset covering three chapters, with the remainder left as future work.
No infrastructure budget	Deployment had to fit within free hosting tiers, which constrained memory use and forced the parsed knowledge base to be cached at build time rather than rebuilt on startup.
Verification against the source	Because no ground-truth dataset existed, correctness had to be established by comparison against real pages of the document, which set the pace of the data understanding phase.

1.5 Proposed Solution

1.5.1 General Presentation of the Solution

The proposed system separates two things that generic assistants combine: deciding what the facts are, and deciding how to say them. Facts come from a structured knowledge base built by parsing the document once, offline, with rules derived from the document's own geometry and validated against printed pages. Phrasing comes from a language model, which receives facts that have already been retrieved and is restricted to rewording them.

Between those two layers sits a verification step. Before any generated sentence reaches a user, it is compared against the retrieved facts, and any answer that has dropped or altered a role name is rejected in favour of a deterministic template. The user still gets an answer, and the answer is still correct; it is simply less conversational. This is the mechanism that makes the language model safe to use at all in this setting, and it is described in detail in Chapter 4.

1.5.2 Global System Architecture

The system is organised in four layers. An offline extraction layer converts the source PDF into a validated node set, which is cached and shipped with the application. A knowledge layer holds that node set as a directed graph and provides lookup and search over it. An agent layer interprets questions, resolves them to activities, constructs deterministic answers, and applies the language model and its verification step. A presentation layer exposes the agent through a REST interface and an authenticated web client offering conversation, document attachment, export, sharing, and analytics.

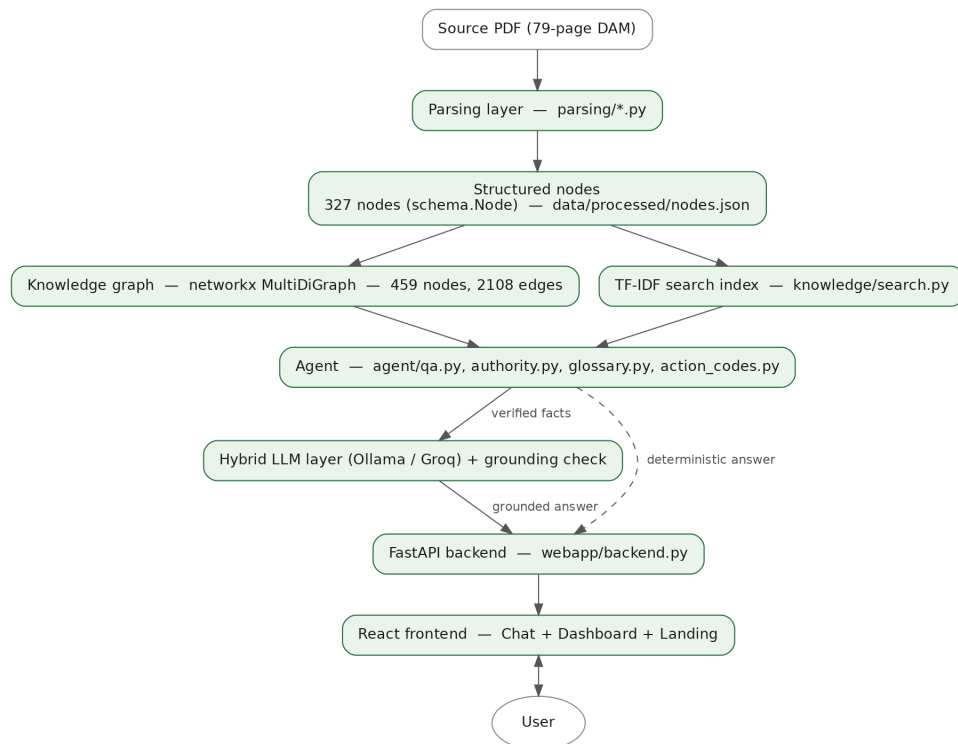


Figure 1.1: Global architecture of the proposed system, from source document to user-facing answer.

One consequence of this separation is worth noting early. Because extraction happens offline and its output is cached, the running application never touches the PDF. It loads a prepared node set at startup, holds it in memory, and answers from

it. That decision was made for deployment reasons, and Chapter 5 explains what happened when it was not made.

1.5.3 Selected Technologies

Table 1.5: Selected technologies and their role in the system.

Technology	Role in the system
Python	Language for extraction, knowledge base, agent, and backend.
pdfplumber	Character- and word-level PDF extraction with positional coordinates, which is what makes geometric reasoning about rotated headers and superscripts possible.
Pydantic	Schema definition and validation for the node model, so malformed records fail at construction rather than at query time.
networkx	Knowledge graph construction and traversal, using a directed multigraph so that a role can hold several distinct responsibilities over the same activity.
scikit-learn	TF-IDF vectorisation and cosine similarity for free-text activity resolution.
FastAPI	REST backend, chosen for its typed request and response models and automatic API documentation.
React and Vite	Web client, built as a single-page application and served as static files by the backend.
Groq and Ollama	Language model providers for answer phrasing, cloud-hosted and local respectively, behind a common interface so either can be used or neither.
Web Speech API	Browser-native speech recognition for voice input, requiring no server-side audio processing.
python-jose and passlib	JSON Web Token issuing and password hashing for the authentication layer.
ReportLab	Server-side PDF generation for conversation export.
statsmodels	Time-series decomposition and forecasting for the analytics layer.
Docker	Containerisation, so the application runs identically in development and in deployment.

Technology	Role in the system
Render	Hosting platform, used on its free tier.

Conclusion

The DAM answers an important question, and the way it is published makes that answer expensive to retrieve. Manual consultation is slow and error-prone because the document's meaning lives in its layout, and generic document assistants fail for the same reason, with the added risk that they fail fluently. The requirements set out above respond to both problems, and the architecture proposed in response separates fact retrieval from phrasing so that the flexibility of a language model can be used without inheriting its unreliability. The next chapter turns to the source document itself and examines, in detail, what makes extracting it difficult.

Chapter 2

Data Understanding: Analysis of the DAM Source Document

Introduction

In most data science projects this chapter would describe a dataset: rows, columns, distributions, missing values. Here the raw material is a PDF, and the equivalent exercise is to understand how a printed page encodes information that no field in a table ever declares. The structure exists, but it is visual. A column belongs to a role because of where it sits on the page, not because anything in the file says so.

This chapter therefore does two things. It describes what the document contains and how that content is organised, and it documents the specific ways automated extraction goes wrong on it. The second part matters as much as the first. Every parsing rule described in Chapter 3 exists because of a failure observed here, and each of those failures was found by comparing extracted output against a real page rather than by reasoning about what the extraction ought to produce.

2.1 Source Document Presentation

2.1.1 Source and Context

The source is the African Development Bank Group's Delegation of Authority Matrix, an internal reference document of more than two hundred and fifty pages distributed as a PDF. It is not a published research dataset and has no accompanying schema, documentation, or machine-readable export. What exists is the document as staff receive it.

Work was scoped to a seventy-nine-page subset covering three chapters. An earlier and narrower export of sixty-six pages was used at the very beginning of the project and then replaced, for two reasons. The larger file includes the glossary, the abbreviations list, and the authority-code legend, all of which the system needs as reference data rather than as content to be parsed. It also contains a full chapter on Non-Sovereign Operations that the initial scope had excluded. That chapter was

brought into scope once its table structure was examined and found to be compatible with the same parsing approach, at the cost of one substantial new class of layout problem described in Section 2.3.5.

2.1.2 Hierarchical Structure of the Document

The DAM is organised as a four-level hierarchy, and recovering that hierarchy correctly turned out to be one of the more consequential parts of the work.

At the top are chapters, which group the Bank's activities into broad domains. Within a chapter are processes, identified by a rounded number such as 2.510, which group related activities. Within a process are tasks, identified by sequential numbers such as 2.513. Some tasks decompose further into child tasks, identified by a third number component such as 1.117.2. Finally, a small but important category of items are threshold variants: sub-items of a task keyed not by a sequence number but by a monetary band, expressing that the answer to "who approves this" changes depending on the amount involved.

Identifier shape does not reliably indicate level, which is a trap worth stating explicitly. A task belongs to a process not because its identifier begins with the process identifier as a string, but because of arithmetic: tasks are grouped into a process by rounding down to the nearest ten. Task 2.513 belongs to process 2.510, and the string "2.513" does not begin with "2.510". Any implementation that derives the hierarchy by prefix matching gets two of the four levels wrong while appearing to work on the other two, which is precisely what happened in an early version and is discussed in Section 3.5.

2.1.3 Table Layout and the Authority Notation

Content is presented as wide tables. The leftmost columns carry the activity identifier and its title. The remaining columns, of which there may be a dozen or more, each correspond to an organisational role, and their headers are rotated ninety degrees so that long role names fit within the available width.

Each cell at the intersection of an activity and a role carries an authority code, or nothing at all if that role has no part in that activity. Alongside the codes, superscript digits refer to footnotes printed at the bottom of the page, qualifying the responsibility with conditions such as delegation limits or specific circumstances.

Two properties of this presentation create most of the difficulty. The codes are short, so distinguishing a code from a footnote marker cannot rely on length. And the association between a code and a role is purely positional, established by horizontal alignment between a cell and a header printed sideways somewhere above it.

2.2 Exploratory Analysis of the Document

2.2.1 Global Overview and Node Distribution

Once parsed and validated, the seventy-nine-page subset yields 327 structured records, distributed across the four hierarchy levels as shown in Table 2.1. These records carry 1,776 individual responsibility entries, each one an assertion that a named role holds a specific authority code over a specific activity.

Table 2.1: Distribution of node types across the processed subset.

Node type	Count	Description
Chapter	3	Top-level domains of Bank activity covered by the processed subset.
Process	35	Groups of related activities within a chapter.
Task	138	Individual activities, the level most questions are about.
Child task	102	Sub-activities of a task.
Threshold variant	49	Variants of a task keyed by a monetary threshold band.
Total	327	

The proportion of threshold variants is higher than an outside reader might expect, and it reflects something real about how the Bank delegates authority. Approval rights are frequently banded by amount, so a single named activity can carry three or four different approvers depending on the sum at stake. A system that treated each activity as having one answer would be wrong for roughly fifteen percent of the document.

2.2.2 Authority Code Distribution

Across the 1,776 responsibility entries, the codes are not evenly distributed. Informed markers and check-and-verify obligations are considerably more common than approvals, which is unsurprising: a single approval is usually surrounded by several roles that verify the work beforehand and several more that must be notified afterwards.

This distribution has a direct design consequence. If a user asks who approves an activity and the system answers with only the approver, it returns the smallest part of the picture and omits obligations that are mandatory rather than contextual. That

observation is where requirement FR4 came from, and the behaviour it specifies is implemented in Chapter 4.

2.2.3 Role and Actor Analysis

The document names 131 distinct roles across the processed subset, ranging from country office staff through sector managers, division managers, directors, and vice-presidents up to the Board. Role names are long and heavily abbreviated, which is what forces the rotated headers in the first place, and the abbreviations are not always defined in the document's own glossary.

That last point produced a small but visible feature of the finished system. Some acronyms appear throughout the role names without ever being separately defined in the abbreviations pages. Rather than leaving a user to guess, the system maintains a small set of curated aliases for these, and it states plainly when an expansion comes from that curated set instead of from the document's own glossary.

Averaged across all 327 records, each activity carries 5.48 responsibility entries. Just under thirty percent of records carry no direct responsibilities at all, which at first looks like an extraction failure and mostly is not. Chapters and processes are organisational containers rather than activities, so they legitimately have none, and a further group of tasks are pure cross-references, discussed below.

2.2.4 Threshold Variants and Monetary Bands

Threshold variants were not part of the initial understanding of the document's structure. They were discovered during extraction, when rows appeared that had the shape of a task's sub-item but carried a monetary band where a sequence number was expected.

Recognising them as a distinct level rather than forcing them into the child-task category mattered for correctness. A child task is a component of a task, and both may be performed. A threshold variant is an alternative to its siblings: exactly one applies to any given case, selected by the amount involved. Collapsing the two would have produced answers listing several approvers for a single transaction, all of them real in the document and only one of them applicable.

2.2.5 Cross-References and Redirects

A number of rows in the DAM record no responsibilities and instead contain a pointer to another part of the document. These take two forms. The first is an explicit list, phrased as "See DAM 16.100, 16.200" and similar. The second is a range, phrased

as “Refer to Activities 2.114 – 2.117 in Section 2”, which must be expanded to every identifier it covers rather than treated as two endpoints.

Across the processed subset, fourteen such references were captured. A human reader handles these by turning to the referenced page and starting again. The system was built to follow them automatically, because a user who asks about activity 1.117.2 and is told that no responsibilities are recorded has been given a technically accurate and practically useless answer, when the document itself says where to look.

2.2.6 Consolidated View

Taken together, the exploratory analysis establishes several things that shape everything downstream. The document is hierarchical but its hierarchy cannot be derived from identifier strings. Its meaning is positional, carried by alignment between cells and rotated headers. A significant minority of its rows are not activities at all but pointers or threshold alternatives. And its responsibility entries are dominated by obligations that a naive question-answering system would discard as secondary.

2.3 Extraction Quality Assessment

This section documents what goes wrong when the document is processed automatically. Each issue below was identified by comparing extracted output against the printed page it came from. That method is slower than checking whether the code runs without error, and it is the only method that finds problems of this kind, because every one of these failures produces output that looks entirely reasonable until it is compared with the source.

2.3.1 Text Layer Inspection

PDF is a page description format. It records which glyph is drawn at which coordinate, with what font and size, and nothing about what those glyphs collectively mean. A library such as pdfplumber recovers characters and words along with their positions, but reconstructing rows, columns, reading order, and cell boundaries is left entirely to the application.

The practical consequence is that every structural property of the document has to be rediscovered from geometry. A row is not a row in the file; it is a set of words whose vertical positions are close enough to be considered aligned. How close is close enough is not stated anywhere and had to be measured.

2.3.2 Rotated Column Headers

Role headers are printed at ninety degrees. Standard word extraction returns them as ordinary text with ordinary coordinates, giving no indication that they are rotated, and their reported horizontal position corresponds to the narrow strip the rotated text occupies rather than to the full column beneath it.

Attributing a code to a role therefore requires clustering header text by its horizontal position and matching each cell against the nearest header cluster. A complication appeared partway through the document: on some pages the header row is printed once and the table continues onto following pages without repeating it. Those continuation pages have codes and no headers at all, and attribution fails completely unless the last seen header set is carried forward.

2.3.3 Fused Footnote Digits and Authority Codes

Footnote markers are printed as superscript digits immediately adjacent to authority codes. After extraction, a code such as A followed by footnote 3 arrives as the string “A3”, which is indistinguishable by shape from the legitimate level-bearing code A3.

Distinguishing them requires the vertical offset and font size of the digit relative to the letter, both of which pdfplumber reports per character. A superscript sits higher and is set smaller. The rule that separates the two cases was derived by measuring real characters on real pages rather than assumed, because the margins involved are small and a threshold guessed from intuition would have misclassified a substantial number of codes.

A related defect affected the informed marker. The parenthesised informed character had been matched by scanning for a consecutive run of the characters “(”, “i”, “)”. On real pages that marker is rendered as five separate characters, so the scan never matched anything. Before this was found, 174 of the 276 records in an early version of the extraction carried an informed marker in the printed document with no role ever attached to it in the extracted data. The failure was invisible in every output that did not compare records against the page.

2.3.4 Cross-Row Text Bleeding and Title Corruption

Activity titles wrap across lines, and in a dense table a wrapped title line can end up physically closer to the next activity’s row than to its own. Where this happens, the title is attached to the wrong activity. Both activities then look plausible in isolation: one has a slightly short title, the other a slightly long one, and nothing flags an error.

At its worst this affected roughly a quarter of the records produced by an early version of the parser. It was corrected by making title attachment conditional on the state of the parse rather than on proximity alone, using three signals: whether a task is currently open, whether the open task's text terminates with a full stop, and whether the incoming line carries authority codes when the open task already has its own. After those rules were applied, one residual case remained across the whole subset, on activity 2.222.1, and it is documented rather than silently accepted.

2.3.5 Split Rows in the Non-Sovereign Operations Chapter

The Non-Sovereign Operations chapter, added to scope after the initial parser was working, introduced a layout variation that broke row reconstruction. In that chapter a single logical row is sometimes rendered as two physically separated bands, with the authority codes drawn slightly below the activity text rather than aligned to it.

The first attempted fix was a general one: merge any two vertically adjacent bands whose separation falls below a tolerance. Checking it before adopting it showed 1,614 candidate merges across the document, several of which were clearly wrong and would have fused unrelated activities. The fix was rejected and replaced by a narrower rule that merges a band only when its content consists entirely of authority codes, which is a strong signal that it belongs to the row above and cannot be an activity in its own right.

2.3.6 Consolidated Data Issues Summary

Table 2.2: Consolidated summary of extraction issues found in the source document.

Issue	Effect if unhandled	Resolution
Rotated column headers	Codes cannot be attributed to roles.	Cluster header text by horizontal position; carry the header set forward across continuation pages.
Missing headers on continuation pages	Whole pages of responsibilities lost.	Reuse the last seen header set until a new one appears.
Footnote digits fused with codes	Codes misread; footnote qualifications lost.	Separate by measured vertical offset and font size of the character.

Issue	Effect if unhandled	Resolution
Informed marker not matched	Informed parties absent from the majority of records.	Match the marker as the separate characters it is actually rendered as.
Title bleeding across rows	Titles attached to the wrong activity; free-text search degraded.	Conditional attachment based on parse state, terminating punctuation, and code presence.
Split rows in the NSO chapter	Responsibilities detached from their activity.	Merge a band only when it consists entirely of authority codes.
Hierarchy from string prefixes	Two of four hierarchy levels silently wrong.	Derive parentage from explicit parent fields and process arithmetic.
Cross-reference rows	Activities reported as having no responsibilities.	Extract references, expand ranges, and follow pointers at query time.

Conclusion

The DAM is a well-organised document for a human reader and an adversarial one for an automated parser, because every structural property it has is expressed through layout. The issues catalogued in this chapter are not edge cases; between them they affected the hierarchy, the role attribution, the titles, and the informed parties, which is to say every component of an answer the system is expected to give.

What the chapter also establishes is a working method. None of these problems was found by reading code or by reasoning about what the parser should produce. Each was found by taking extracted output and comparing it against the page it claimed to represent. That discipline carries through the rest of the thesis, and Chapter 3 describes the pipeline built on top of these findings.

Chapter 3

Data Engineering Pipeline and Knowledge Graph Construction

Introduction

This chapter describes how the source document becomes queryable data. It covers the processing environment and why it was chosen, the design of the extraction pipeline and the transformations applied at each stage, the validated node set that results, and the knowledge graph built on top of it. It closes with an assessment of the pipeline's performance and of the limitations that remain in its output.

3.1 Processing Environment

3.1.1 Motivation for a Deterministic, Rule-Based Pipeline

An alternative approach to this extraction problem would be to train a model: annotate a sample of pages, learn to classify text regions, and generalise. It was rejected for reasons that were practical before they were principled. No annotated data existed, and producing enough of it for a single seventy-nine-page document would have consumed more effort than writing rules, while leaving the result harder to verify.

Verifiability is the principled reason. A rule derived from measured character geometry can be checked against the page that motivated it, and when it fails, the failure is inspectable and the fix is local. A learned region classifier that misreads one table gives no comparable account of itself. For a document where errors are silent and plausible, being able to explain why a given character was interpreted the way it was is worth more than the flexibility a learned approach would add.

3.1.2 Why Not a Distributed Processing Framework

Distributed frameworks such as Apache Spark are the standard answer when a pipeline must process volumes that exceed a single machine. That is not the situation here. The full document is a few megabytes, the processed subset yields 327 records,

and a complete rebuild of the knowledge base takes roughly fifteen to twenty seconds on an ordinary laptop.

Introducing a distributed framework would have added an execution model, a cluster configuration, and a serialisation boundary to a problem that has none of the characteristics those things exist to address. The difficulty in this project is interpretive rather than computational: the hard part is deciding what a character means, not processing many of them quickly. The pipeline is therefore a single-pass, single-process program, which is the right shape for the problem and also the shape that makes it easy to inspect intermediate output during development.

3.1.3 Tooling

Extraction uses `pdfplumber`, which exposes characters and words with their coordinates, font names, and sizes. That character-level access is not a convenience but a requirement, since the superscript rule described in Chapter 2 depends on per-character vertical offset and size and cannot be implemented on word-level output alone.

The node model is defined with `Pydantic`, so that structural expectations are enforced at construction time. A record missing a required field, or carrying an authority code outside the known set, fails immediately and loudly rather than propagating into the graph and surfacing later as a strange answer.

3.2 Pipeline Design

3.2.1 Global Architecture

The pipeline is a sequence of eight stages, each consuming the output of the previous one and each independently inspectable. Raw characters become physical rows, rows become task blocks, task blocks are cleaned and enriched with geometric and role information, and the enriched blocks are assembled into validated records and merged with the document's structural skeleton.

Table 3.1: Stages of the data engineering pipeline.

Stage	Name	Function
1	Raw extraction	Characters and words with coordinates, fonts, and sizes are read from the PDF.

Stage	Name	Function
2	Row reconstruction	Words are grouped into physical rows by vertical alignment, with split rows merged where the merge is provably safe.
3	Task block detection	Rows are grouped into task blocks, and boundaries between consecutive activities are determined.
4	Title and reference cleaning	Titles are normalised, cross-references are extracted and expanded, footnote digits are stripped from title text.
5	Geometric attribution	Authority codes are separated from footnote markers by character geometry, and levels are attached.
6	Role attribution	Codes are matched to roles by horizontal alignment with clustered, rotated column headers.
7	Node assembly	Enriched blocks are validated against the schema and merged with the chapter and process skeleton.
8	Validation	Whole-document consistency checks are run across the assembled node set.

3.2.2 Data Flow and Modular Organisation

Each stage lives in its own module, and the boundaries between them were drawn so that a failure can be localised by inspecting the output of one stage rather than by tracing execution through the whole program. During development this mattered constantly: when a record came out wrong, the question was always which stage first produced something incorrect, and that question is only cheap to answer if the intermediate results are inspectable.

Parsing modules handle rows, task blocks, metadata, action geometry, column roles, and text cleaning. Modelling modules handle node construction, the structural skeleton, the graph, and the cache. The schema sits in its own module, depended on by both groups and depending on neither.

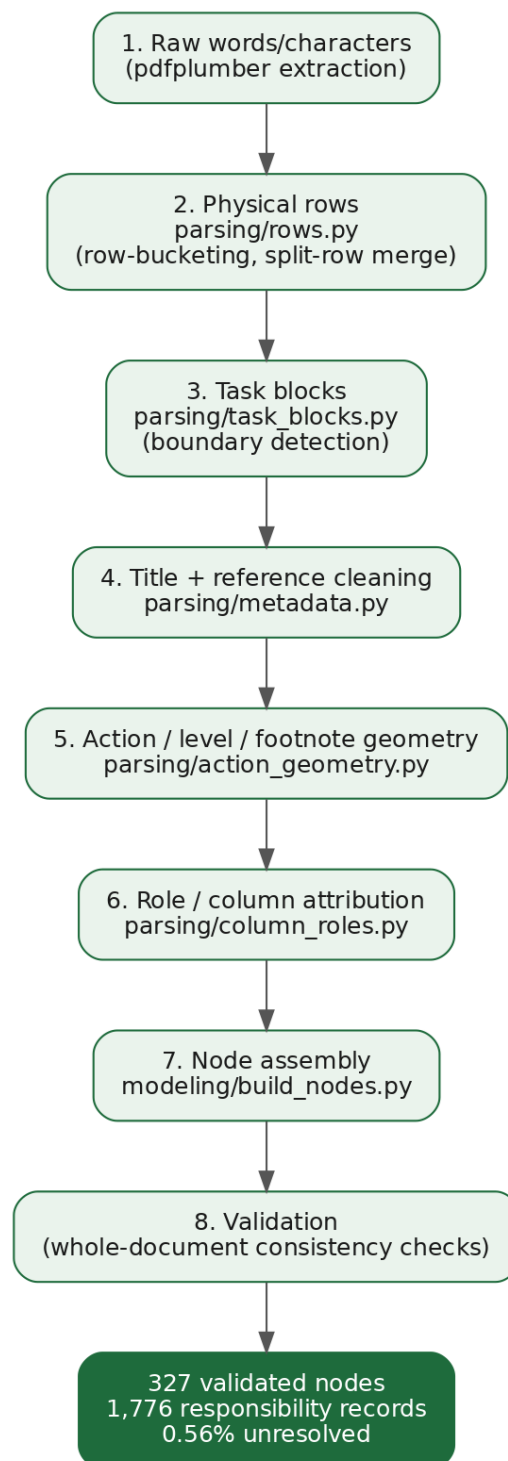


Figure 3.1: The eight-stage data engineering pipeline, from raw PDF extraction to a validated node set.

3.3 Data Ingestion

Ingestion walks the document page by page. For each page, characters are read with their coordinates and font metrics, and words are read with their bounding boxes. Both are retained, since different stages need different granularity: row reconstruction works on words, while the superscript rule works on characters.

Reference data is ingested separately and from different pages of the same document. The authority code legend is transcribed into a structured file listing each full code, its level, and its meaning, and validated against the schema so that all seventeen entries load cleanly. The abbreviations pages are parsed into a glossary, which required its own handling: entries wrap across lines, some rows are section headings rather than definitions, and a term and its definition are occasionally rendered a fraction of a point out of alignment, so row grouping needs the same tolerance used elsewhere in the pipeline.

3.4 Document Processing and Normalisation

3.4.1 Row Reconstruction

Words are grouped into rows by clustering their vertical positions. A tolerance is applied, because text that is visually on the same line is frequently not at an identical coordinate. Where the Non-Sovereign Operations chapter splits a logical row into two bands, the two are merged under the restricted condition described in Chapter 2: only when the lower band consists entirely of authority codes.

3.4.2 Task Block Detection

Rows are then grouped into blocks, each corresponding to one activity. Boundary detection is the part of the pipeline that required the most iteration, because the visual cue for a boundary is inconsistent. Three signals are used together: whether a task is currently open, whether the open task's accumulated text ends with a full stop, and whether an incoming row carries authority codes when the open task already has some of its own. The third signal is the strongest, since a second batch of codes almost always indicates that the next activity has begun.

3.4.3 Title and Reference Cleaning

Titles are normalised by stripping footnote digits, collapsing whitespace, and removing cross-reference phrases that would otherwise be absorbed into the title as

noise. Reference extraction is centralised in a single function used by every path that touches titles or notes. This was not the original design: two divergent implementations existed at one point, one of which handled a reference pattern the other did not, and the inconsistency produced titles that differed depending on which path had processed them.

3.4.4 Geometric Attribution of Authority Codes

Authority codes are identified within each row and separated from footnote markers by the character-level rule described in Chapter 2. Level digits belonging to a code are retained as part of it; superscript digits are recorded separately as footnote reference numbers attached to that responsibility.

3.4.5 Role Attribution

Role headers are extracted, clustered by horizontal position, and associated with the column region they label. Each authority code found in a row is then matched to the header cluster whose horizontal extent contains it. Where a page carries no headers because the table has continued from a previous page, the last seen header set is carried forward.

Roles that cannot be matched to any header are recorded as unresolved rather than discarded or guessed. Keeping them visible is deliberate: an unresolved role is a known gap that can be counted and reported, whereas a discarded one is an invisible loss.

3.5 Node Construction and Validation

Enriched blocks are assembled into records conforming to the schema, and merged with a structural skeleton that supplies the chapter and process nodes, since those are organisational containers that the table parser never emits as activities.

Parentage is derived explicitly rather than inferred from identifier shape. Each record carries pointers to its parent, and the hierarchy is reconstructed from those pointers. An earlier implementation derived children by string prefix matching, which happens to work for two of the four levels and fails for the other two, and the failure was invisible until threshold variants entered the identifier space and a task's grandchildren began colliding with its direct children under the same prefix. The bug was caught by a consistency check requiring that every record's children be of the expected type and that every child's own parent pointer point back, rather than by inspection.

Table 3.2: Data quality indicators after node construction.

Indicator	Value
Structured records produced	327
Responsibility entries extracted	1,776
Distinct roles identified	131
Unresolved role entries	10 (0.56%)
Average responsibilities per record	5.48
Records with no direct responsibilities	29.63%
Cross-references captured	14
Responsibility entries carrying footnote references	113 (6.36%), across 44 records
Residual title corruption	1 record (2.222.1)

3.6 Knowledge Graph Modelling

3.6.1 Why a Graph

The validated node set could have been stored as relational tables or as a flat document collection. A graph was chosen because the questions the system must answer are relationship questions. “Who approves this activity” is a traversal from an activity node along edges of a particular type, and “which activities does this role approve” is the same traversal in reverse. Both are single operations on a graph and joins or scans on a flat structure.

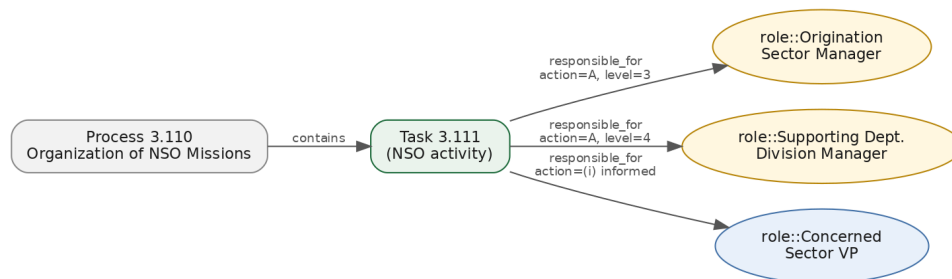
A directed multigraph is used rather than a simple graph, because a role can hold more than one distinct responsibility over the same activity. Two nodes may therefore be connected by several parallel edges, each carrying its own authority code and footnote references.

3.6.2 Node and Edge Schema

The graph contains two kinds of node. Activity nodes correspond one-to-one with the 327 structured records. Role nodes correspond to the 131 distinct roles. Edges carry the relationships between them.

Table 3.3: Edge types in the knowledge graph.

Edge type	From → To	Meaning
contains	Activity → Activity	Hierarchical containment: chapter to process, process to task, task to child task or threshold variant.
responsible_for	Role → Activity	A recorded responsibility, carrying the authority code, its level, and any footnote references.
references	Activity → Activity	A cross-reference extracted from the activity's own text, including ranges expanded to their members.

**Figure 3.2:** Excerpt of the knowledge graph around activity 3.111, showing the containment edge from its parent process and the responsibility edges connecting it to its approving and informed roles.

3.6.3 Graph Statistics

The assembled graph holds 459 nodes and 2,108 edges. The node count is the sum of the 327 activities and the 131 roles, plus one aggregate node. The edge count is dominated by responsibility edges, with containment edges accounting for the hierarchy and a small number of reference edges added from cross-references.

3.7 Pipeline Evaluation

3.7.1 Execution and Caching

A full rebuild from the PDF takes roughly fifteen to twenty seconds. That is acceptable offline and unacceptable per request, so the assembled node set is serialised to a cache file at build time and loaded at application startup. The running service never opens the PDF.

This was not the original arrangement, and the reason it changed is covered in Chapter 5: the first deployment attempt parsed the document during startup and was terminated by the hosting platform for exceeding its memory limit. Caching resolved that, and it also removed a subtler risk, since parsing at startup would have made the served content depend on the runtime environment rather than on a verified build artefact.

3.7.2 Data Quality Outcomes

The unresolved-role rate of 0.56 percent is the pipeline's headline quality figure, and it should be read carefully. It measures the proportion of extracted responsibility entries whose role could not be matched to a column header. It does not measure whether the correct code was extracted for every cell, which no automated check can establish in the absence of ground truth. That second question was addressed by sampling: specific activities were compared entry by entry against the printed page, including activities chosen precisely because they had previously been extracted incorrectly.

3.7.3 Observed Limitations

Two records in the processed subset, 2.516 and 2.522, have empty titles that the parser could not recover, and one record, 2.222.1, retains a residual title corruption. All three are known and documented rather than quietly accepted. A candidate fix for the last case was implemented, tested, found not to resolve the target case, found to introduce a regression elsewhere, and reverted. Leaving a documented defect in place was judged better than carrying a change that made the overall extraction worse.

A second limitation concerns footnotes and is discussed further in the general conclusion. Footnote reference numbers are extracted and attached to responsibilities, but the footnote text printed at the foot of each page is not yet parsed, so the system can report that a qualification applies without stating what it says. Two related extraction defects were identified during testing: on activities 1.116.1 and 1.116.2 a column header was mistaken for a role name, and on the same records two adjacent footnote digits were fused into a single number. Both are documented in Chapter 5 alongside the planned remediation.

The pipeline has also only been exercised against three chapters. The remaining chapters of the full document use the same table conventions as far as can be determined, but that expectation is untested, and processing them would be the first step of any continuation of this work.

Conclusion

The pipeline converts a document whose structure is purely visual into 327 validated records carrying 1,776 responsibility entries, with under one percent of those entries unresolved, and turns that output into a graph of 459 nodes and 2,108 edges that answers relationship questions by traversal. Each stage is deterministic and inspectable, which is what allowed the defects described in Chapter 2 to be found, fixed, and prevented from recurring. The next chapter builds the agent that queries this graph.

Chapter 4

Retrieval, Grounded Generation and Agent Design

Introduction

The knowledge graph holds the facts. This chapter describes the layer that turns a typed or spoken question into an answer drawn from it. Three problems have to be solved in sequence: working out which activity the user means, working out what they want to know about it, and saying it back in a way that reads naturally without becoming unfaithful in the process.

The third problem is where most of the design effort went, and it is the one that distinguishes this system from a conventional retrieval-augmented assistant. The language model here is not trusted to decide anything. It receives facts that have already been retrieved and is permitted only to rephrase them, and what it produces is verified against those facts before it is shown.

4.1 Retrieval Methodology

4.1.1 Identifier Fast Path

When a question contains something shaped like an activity identifier, the system tries a direct lookup first. Identifiers are unique, so a match is exact and carries no uncertainty. This path is cheap and it is the one that serves users who already know what they are looking for.

An identifier that is not in the node set is treated as a distinct outcome rather than as a failed lookup. This distinction matters more than it sounds. In an earlier version, an unrecognised identifier simply fell through to free-text search over the whole question, including the bogus identifier itself. The remaining words in the question, things like “task”, “who”, and “approves”, share enough vocabulary with real activity titles to score above zero, so the system would return a confident answer about an unrelated activity while the user believed they were being told about the

identifier they had asked about. The behaviour was reachable with ordinary phrasing, not a contrived edge case.

4.1.2 Free-Text Resolution

Questions that name an activity rather than identifying it are resolved by lexical similarity. Activity titles are vectorised with TF-IDF and compared against the question by cosine similarity, and the highest-scoring node is returned along with its score.

Process-level titles are included in the index alongside tasks, child tasks, and threshold variants. They were excluded at first, on the reasoning that users ask about activities rather than about organisational groupings. That reasoning was wrong in a specific way: a user asking about the organisation of missions in the non-sovereign chapter was matching nothing useful, because the only node whose title actually contained that phrasing was a process and processes were not indexed.

4.1.3 Why Lexical Retrieval Rather Than Embeddings

Semantic retrieval with learned embeddings is the more fashionable choice and would be the right one for a large, varied corpus written in ordinary language. This corpus is neither large nor varied. It is a few hundred short titles written in a controlled institutional vocabulary, where the words a user is likely to type are mostly the words the document already uses.

Lexical matching also has a property that matters for this project: its behaviour is explainable. A score can be traced to shared terms, a mismatch can be diagnosed by looking at the vocabulary, and the whole layer runs with no model download, no inference cost, and no dependency on an external service. Given that retrieval accuracy is the foundation everything else rests on, the ability to reason about why a given node was returned was worth more than the additional recall an embedding model might have offered.

4.1.4 Typo Tolerance

People misspell things, particularly long institutional phrases. Rather than relying on a language model to interpret intent, the system applies a deterministic correction step: each alphabetic word in the question is compared against the vocabulary the document itself uses, and replaced with the closest match when the similarity exceeds a threshold and the word is not already an exact match.

The threshold was calibrated against a real false positive rather than chosen by intuition. Set too low, ordinary words get rewritten into document vocabulary they merely resemble, which corrupts questions that were correct to begin with. Because the step is deterministic, it behaves identically whether or not a language model provider is reachable.

4.2 Evaluation Protocol and Confidence Semantics

There is no held-out test set here, because there is no labelled corpus to hold out. Correctness is established instead by comparison against the source document, and every resolution carries a method label and a confidence value so that the basis of an answer is visible to the user rather than implied.

Table 4.1: Resolution methods and their confidence semantics.

Method	Confidence	Interpretation
id	1.0	Exact identifier match. No ambiguity.
text_search	TF-IDF score	Resolved from the activity's name. The score is shown, and low scores are flagged in the interface.
invalid_id	—	An identifier was recognised in the question but does not exist in the document.
needs_clarification	—	The question is too vague to resolve to a single activity.
out_of_scope	—	The question is not about the subject matter of the document.
glossary	—	The question asked for the expansion of an abbreviation.
action_code_legend	—	The question asked what an authority code means.
smalltalk	—	Conversational input with no lookup component.

Exposing the method alongside the answer is a small decision with a large effect on trust. A user who can see that an answer came from an exact identifier match treats it differently from one resolved by name at a middling similarity score, and that difference in treatment is appropriate.

4.3 Deterministic Answer Construction

4.3.1 Intent Detection

Once an activity is resolved, the system determines what is being asked about it. Questions map onto the document's authority codes, and the mapping has to accommodate the way people actually phrase things: "who signs off on this" means approval, "who has to check this" means check and verify.

The mapping is deliberately not naive about the letter C. As set out in Chapter 1, C1 and C2 mean check and verify while C3 and C4 mean consult, and treating the word "check" as equivalent to the bare letter C would merge two unrelated obligations. Intent detection therefore works against full codes rather than their first character.

4.3.2 Mandatory Obligation Surfacing

An answer reports what was asked and then adds what the document makes mandatory regardless. If an activity has a recorded check-and-verify obligation, that obligation is stated even when the question was about approval. If informed parties are recorded, they are named.

The reasoning is that these are not contextual extras. A check-and-verify entry in the DAM is an institutional requirement, and a new employee asking a narrow question is exactly the person who does not know to ask the other three. The note is suppressed only when it would duplicate the answer already given.

4.3.3 Cross-Reference Following

Where a resolved activity has no responsibilities of its own but does carry a cross-reference, the system follows the reference and reports the target's responsibilities, stating that it has done so. This mirrors what a human reader does when a row tells them to look elsewhere, and it converts a dead end into an answer.

4.3.4 Honest Refusal

Three cases produce an explicit non-answer. An identifier that does not exist is reported as not existing, with a suggestion to check it or describe the activity in words. A question too vague to resolve prompts for more detail. A question outside the document's subject matter is declined as out of scope.

None of these paths is allowed to degrade into a best guess. This is the behaviour that requirement FR7 specifies and the property that gives the system's positive answers their value: a tool that answers everything tells you nothing about which of its answers to trust.

4.4 Auxiliary Query Types

Not every question is an authority lookup. Three additional categories are handled directly.

Glossary questions ask what an abbreviation means and are answered from the abbreviations parsed out of the document, supplemented by a small curated set of aliases for acronyms that appear in role names without being separately defined. When an expansion comes from the curated set, the system says so rather than presenting it as sourced from the document.

Authority-code questions ask what a code means and are answered from the transcribed legend. Detection here is deliberately conservative about bare single letters, since “I” collides constantly with the pronoun. A bare code counts as a real mention only when it appears alongside another code or is unambiguous on its own, such as the parenthesised informed marker or a letter-and-digit combination that is not a plausible English word.

Conversational input, greetings and thanks and questions about what the system can do, is matched against anchored patterns and answered directly, short-circuiting the lookup pipeline. The patterns are anchored against the whole message rather than searched as substrings, because a substring test matches “hi” inside “history of the DAM”.

4.5 Hybrid Language Model Layer

Answer phrasing is delegated to a language model through a provider interface with two implementations and four operating modes.

Table 4.2: Language model providers and operating modes.

Mode	Provider	Behaviour
Auto	Cloud, falling back to local, then template	Default. Uses whichever provider is available, and the deterministic template if neither is.

Mode	Provider	Behaviour
Cloud	Groq	Hosted inference. Fast, requires an API key and network access.
Local	Ollama	Runs on the same machine or in a companion container. No external dependency.
Off	None	Template answers only. Every lookup capability remains available.

The interface between the agent and the providers is narrow by design, which paid off during the project when a hosted model was deprecated by its provider and began returning errors on the deployed instance. The fix was a single constant change behind the interface, with no effect on the agent, the retrieval layer, or the tests covering them.

4.6 Grounded Generation and the Verification Check

4.6.1 The Contract Given to the Model

The model never sees the knowledge graph and never chooses what to retrieve. It receives a question, a set of already-retrieved facts, and a system prompt instructing it to state those facts naturally, to invent nothing, to omit nothing, and to avoid formatting of any kind. Where the answer must be produced in another language, an additional rule instructs it to translate the sentence while copying role names, identifiers, and footnote numbers exactly as given.

4.6.2 The Verification Step

A prompt is an instruction, not a guarantee. The verification step is what turns the instruction into a property of the system.

Before a generated answer is returned, every role name in the facts backing that answer is checked for presence in the generated text. If any is missing, the generated answer is discarded and the deterministic template answer is returned in its place. The check is a containment test rather than a similarity judgement, which means it cannot be satisfied by a paraphrase: a model that renames a role, merges two roles into one, or quietly drops the third of four informed parties fails the check, regardless of how fluent the result is.

Comparison is performed after normalising whitespace and case on both sides. That tolerance was added after the check was observed rejecting otherwise faithful

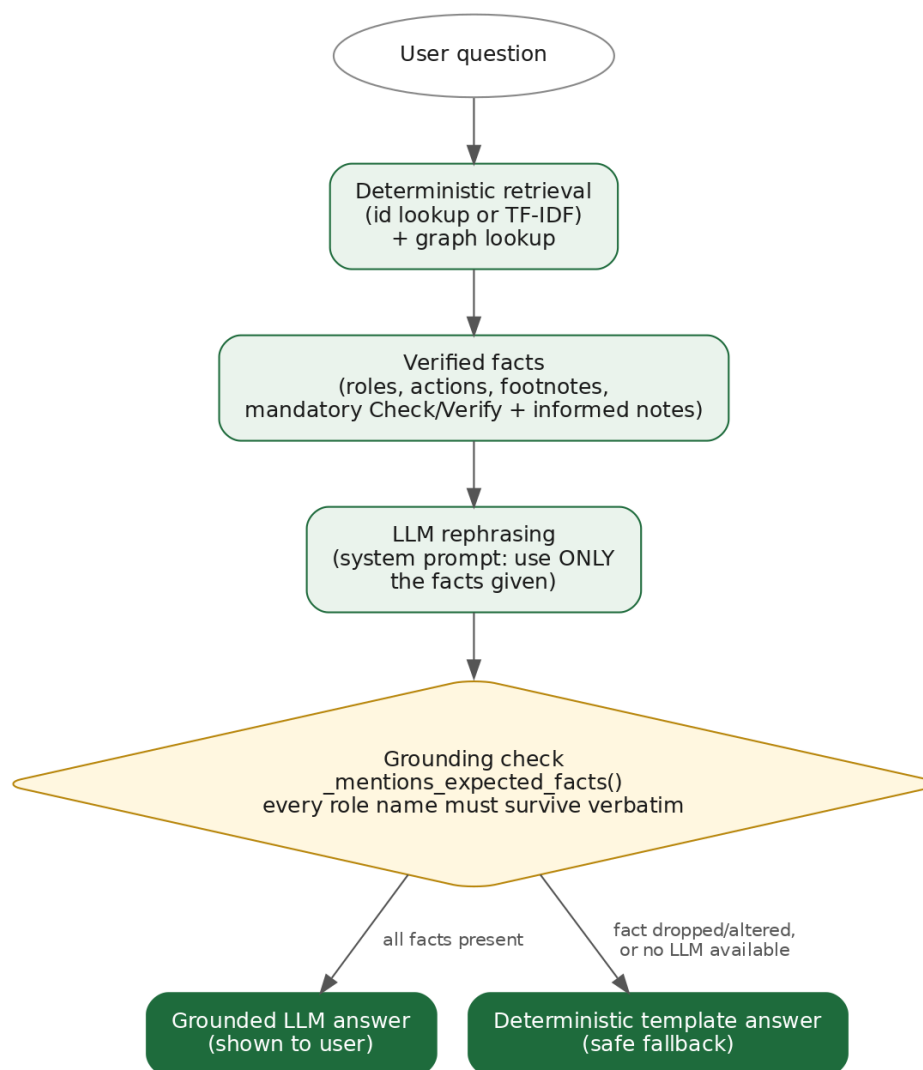


Figure 4.1: The grounded generation flow. The model may only rephrase facts that have already been verified, and any rephrasing that drops or alters a fact is discarded in favour of the deterministic template answer.

rephrasings over nothing but incidental spacing differences. It loosens formatting sensitivity without loosening what is actually verified, since the same words in the same order must still be present.

4.6.3 Answer Length and the Fallback Rate

The verification step initially fired more often than expected, and the cause turned out to be a conflict between two instructions rather than model unreliability. The system prompt asked for an answer in one to three sentences. Activities with many recorded responsibilities cannot be stated completely in three sentences, so the model, following the brevity instruction, dropped facts, and the check correctly rejected the result.

The fix relaxed brevity rather than accuracy. When the number of facts backing an answer reaches a threshold, the brevity rule is replaced with an instruction to use as many sentences as the facts require, while every other constraint stays in place. The fallback rate dropped and the verification criterion was left untouched, which was the point: the accuracy requirement was never the thing that needed adjusting.

4.7 Conversational Capabilities

4.7.1 Context Carry-Over

A user who has just asked about an activity and then asks “and who has to be informed?” expects the second question to be understood against the first. The interface passes the previously resolved activity identifier with each question, and where a follow-up contains no subject of its own, it is answered against that activity, with the interface stating that the subject was carried over.

Carry-over is suppressed for question types that have nothing to do with the previous activity. Authority-code questions were briefly swallowed by it, so that asking what a code means while an activity was in context produced an answer about that activity instead of about the code.

4.7.2 Tone Adaptation

Questions are classified as neutral, frustrated, or confused, and a short empathetic opener is prepended where the tone is not neutral. Classification runs only when a cheap pre-filter suggests it is worth doing, so the common case costs nothing extra, and it is applied to what the user actually typed rather than to any internally translated version of it.

4.7.3 Multilingual Questions

Questions in French, Spanish, Portuguese, or Arabic are detected by a cheap pre-filter based on character ranges and function-word lists, translated into English for retrieval, and answered in the language they were asked in. Retrieval itself remains English-only, which keeps the index and its behaviour unchanged.

The verification check is not translated. It continues to look for the literal English role names inside the answer, whatever language the surrounding sentence is in, which is deliberate: a French answer that dropped a role name fails exactly as an English one would.

4.8 Worked Examples

The following exchanges were run against the delivered build and are reproduced as the system produced them.

Asking *“who approves 2.126”* resolves by identifier with confidence 1.0 to the quarterly mission programme activity, and names the Regional Director General and the Director of the Nigeria office as approvers, with the abbreviation expanded inline. The answer also states that the task must be checked and verified by the Sector Manager and the Supporting Department Division Manager, and that the concerned Sector Vice-President, the Regional Vice-President, and the Task Manager or Project Team must be informed. None of that was asked for. All of it is mandatory.

Asking *“who needs to be informed for quarterly mission program”* resolves the same activity from its name alone, with the answer now centred on the informed parties.

Asking *“who aproves the qaurterly mision program”*, with two deliberate misspellings, resolves identically. The correction is deterministic, so it behaves the same with or without a model provider available.

Asking *“who initiates 1.117.2”* exercises the redirect path. That activity’s own row records no responsibilities and points to another activity, so the system follows the pointer and reports the target’s responsibilities instead of dead-ending.

Asking *“who approves of Communication with Co-Financiers of projects”* followed by *“and who are the informed parties?”* exercises carry-over: the second question stays on the activity resolved by the first.

Asking *“what happens with 9.999.999”* produces a refusal, stating that the identifier does not exist and suggesting the user check it or describe the activity in words. This is the behaviour the design exists to produce, and it is worth reading it as a result rather than as a limitation.

Conclusion

The agent resolves questions through an identifier fast path and a lexical search layer, both deterministic, and constructs answers from records held in the graph. A language model improves how those answers read and is prevented from affecting what they say by a verification step that compares generated text against the retrieved facts and discards anything that has lost one. Around that core sit the behaviours that make the system usable in practice: typo tolerance, redirect following, mandatory obligation surfacing, context carry-over, multilingual handling, and explicit refusal. The next chapter presents the platform built on top of this agent.

Chapter 5

Implementation Results and Delivered Platform

Introduction

This chapter presents what the project delivers: an authenticated web application built on the agent described in Chapter 4, extended with voice input, document attachment, conversation export and sharing, a fully localised interface, and an analytics layer carrying governance indicators and a forecasting capability. It closes with the REST interface, the testing regime, and the deployment arrangement.

5.1 The Delivered Application

5.1.1 Overview and Architecture

The application is a single-page web client served as static files by the same backend that exposes the API. The backend loads the cached node set once at startup, builds the graph and the search index in memory, and answers requests from those structures. The knowledge base itself is read-only and needs no database; a small relational store is introduced only for user accounts, shared conversation snapshots, and the query log, none of which touch the knowledge base.

The web layer holds no domain logic. Every decision about what an answer contains is made in the agent, and the backend wraps it. That separation was maintained deliberately, and it is what allowed the platform features described below to be added without touching the parts of the system that determine correctness.

5.1.2 Authentication and Access Control

Access to the application requires authentication. The mechanism is a token-based flow: credentials are posted to an authentication endpoint, the server verifies them and returns a signed JSON Web Token, and the client presents that token on every subsequent request through an `Authorization` header. Endpoints other than the health check and the login route reject requests without a valid token.

Passwords are never stored in recoverable form. Each account holds a bcrypt hash produced with a per-password salt, and verification compares a hash of the submitted password against the stored value. The signing secret for tokens is supplied through the environment rather than committed, consistent with the existing handling of the language model API key.

Tokens carry the account identifier, the assigned role, and an expiry timestamp, and are signed with HS256. Expiry is set to eight hours, which covers a working day without leaving indefinitely valid credentials in a browser. On the client the token is held in session storage rather than local storage, so it does not survive the tab being closed; this trades a small amount of convenience for a meaningful reduction in exposure, since a token in local storage remains readable by any script that manages to execute on the page.

Two roles are defined. An **analyst** may authenticate, ask questions, attach documents, and export or share their own conversations. An **administrator** additionally reaches the analytics dashboard and the query log. The distinction matters because the analytics layer aggregates what the institution's staff have been asking, which is organisationally sensitive in a way that individual lookups are not.

Two further protections apply to the login route specifically. Failed attempts are rate-limited per source address to slow credential stuffing, and the response to a failed login does not distinguish an unknown account from a wrong password, so the endpoint cannot be used to enumerate valid usernames.

Authentication also enables something the system previously could not do: attribute each question to an account. Every query is recorded with its account identifier, timestamp, resolution method, confidence, and whether the generated phrasing survived the verification step. That record is the foundation of the usage indicators in Section 5.2.3 and the forecasting capability in Section 5.2.4.

5.1.3 The Chat Interface

The chat page is the primary interface. A user types or dictates a question and receives an answer together with the metadata that makes the answer auditable: the resolved activity identifier, how it was resolved, the confidence where one applies, whether a language model was used for phrasing, and which language the answer is in.

Controls in the header select the language model mode described in Table 4.2, including switching it off entirely, and the answer language. Below any answer phrased by a model, a control reveals the underlying deterministic answer, so a user can always see the unembellished version of what the document says. Low-

confidence resolutions are marked visually, conversations are kept in a sidebar, and the layout collapses to a single column on narrow screens.

5.1.4 Voice Input

Questions can be dictated instead of typed. The implementation uses the browser's native speech recognition interface, which means no audio is processed by the application's own backend and no additional infrastructure or inference cost is incurred.

A microphone control sits in the composer. Activating it starts a recognition session configured for a single utterance with interim results enabled, so partial transcription appears in grey as the user speaks and is replaced by the final transcript when the session ends. The recognition language is bound to the interface language selector, so a user working in French dictates in French and a user working in Arabic dictates in Arabic, without a separate setting.

One design decision is worth stating because it was not the obvious one. The transcript populates the input field rather than submitting automatically. Speech recognition performs poorly on institutional acronyms and identifiers, which are precisely what these questions contain, and a user who cannot see the transcription before it is sent has no opportunity to correct a misheard activity number. Populating the field also lets the deterministic typo-correction layer from Chapter 4 act on the transcript, which handles a useful share of recognition errors on activity names without any additional work.

The capability is feature-detected and the control is hidden where the interface is unavailable, which currently affects some browsers. Recognition errors are surfaced explicitly rather than silently: no speech detected, microphone unavailable, and permission denied each produce a distinct message, because a user who has denied microphone permission needs different advice from one whose microphone is not working.

A privacy note belongs in the design rather than in a footnote. The browser interface used here transmits audio to the vendor's own recognition service in most implementations. For an institution handling internal governance material, that is a consideration to weigh consciously, and the honest position is that voice input is a convenience feature whose data path leaves the institution's control. Where that is unacceptable, the same control can be rebound to a locally hosted recognition model at the cost of running that model.

5.1.5 Document Attachment

A conversation can carry an attached document, allowing questions to be asked about material that is not part of the DAM: a draft policy, a related procedure, or a newer extract of the matrix itself.

Uploads are constrained. The endpoint accepts PDF, Word, and plain text files up to ten megabytes, validates the declared type against the file's actual signature rather than trusting the extension, and rejects anything else. The uploaded binary is never persisted and never rendered; text is extracted immediately, the extracted text is retained against the conversation for the life of the session, and the original bytes are discarded. PDF extraction reuses the same library already employed by the offline pipeline, and Word extraction uses a document-object library, so no new class of parsing dependency enters the system.

Retrieval over an attachment reuses the machinery from Chapter 4. The extracted text is split into paragraphs, vectorised with TF-IDF, and searched by cosine similarity, exactly as activity titles are. The retrieved passage is then passed through the same grounded generation and verification path.

The most important part of this feature is a boundary rather than a capability. Answers drawn from an attachment are labelled as such, presented in a visually distinct form, and excluded from the guarantee that applies to DAM answers. The system's central promise is that an answer about the DAM traces to a validated record extracted from a verified document. An uploaded file has been through no such validation; it may be a draft, it may be superseded, and its structure has not been checked against anything. Presenting both classes of answer identically would quietly destroy the property the whole system exists to provide, so the interface states plainly which source an answer came from, and the two are never merged into a single response.

5.1.6 Conversation Export

A conversation can be exported as a PDF document. Generation happens server-side rather than in the browser, which keeps the output identical regardless of the client and allows the export to carry information the on-screen view summarises.

The exported document opens with the institution and programme attribution, the exporting account, the export timestamp, and an identification of the DAM version the answers were drawn from. Each exchange then appears in sequence with the question, the answer as shown, and the provenance attached to it: the resolved activity identifier, the resolution method, the confidence score where one applies, and whether the phrasing was produced by a language model.

Where a language model phrased an answer, the export includes the deterministic template answer alongside it. That choice is what makes the export useful as evidence rather than as a transcript: a reader can see both the readable sentence and the unembellished statement of what the document records, and can check one against the other without access to the running system. For a compliance context, an export that cannot be verified independently is of limited value.

5.1.7 Conversation Sharing

A conversation can be shared with a colleague. Requesting a share produces a signed, expiring token, and the client renders the resulting link as a QR code so that it can be scanned from a phone without retyping anything.

Sharing creates a read-only snapshot. The conversation as it stands is copied server-side and keyed by the token; subsequent messages in the original conversation do not appear in the shared view. The recipient sees the questions, the answers, and the same provenance metadata carried by the export, and can do nothing else with it.

Scope is worth being explicit about, because the obvious reading of the feature is more ambitious than what is described here. This is snapshot sharing, not live collaboration. Two users do not occupy the same conversation, there is no real-time channel between them, and a shared view does not accept new questions. Live multi-party conversation is a different feature with its own state-synchronisation and concurrency problems, and it is not what this delivers.

Three controls apply. Tokens expire after a configurable period, defaulting to seven days. The originating user can revoke a share before expiry. And attached documents are excluded from shared snapshots, since the person sharing a conversation has not necessarily considered that an attachment travels with it.

5.1.8 Interface Localisation

The system already answered questions in five languages. The interface itself did not follow: outside the conversation, the application remained in English, so a user working in Arabic received Arabic answers inside an English page.

The localisation layer resolves that. All interface text is externalised into a message catalogue keyed by string identifier, with one catalogue per supported language. A language provider at the root of the component tree exposes a lookup function to every component, so changing the language re-renders the whole application rather than one page. The active choice persists across sessions, and a missing entry falls back to English rather than displaying a raw key.

Coverage extends to every surface: the landing page, the login form, the chat page, the dashboard including chart axis labels and table headers, the shared conversation view, empty states, error messages, tooltips, and the controls added by the features above. Numbers and dates are formatted through the platform's internationalisation interfaces using the active locale, so a thousands separator or a date order follows the reader's convention rather than the developer's.

Arabic requires more than translation. The document direction attribute is set to right-to-left for Arabic, and the stylesheet uses logical properties, so that margins, padding, and alignment mirror correctly instead of needing a parallel set of rules. The conversation sidebar moves to the opposite edge and chart axes reverse.

One category of text is deliberately never translated. Role names, activity titles, authority codes, and identifiers are reproduced exactly as the source document records them, in every language. They are not interface text but content, and translating them would break both the traceability requirement and the verification mechanism described in Chapter 4, which compares generated answers against the literal role names held in the graph.

5.2 Analytics and Forecasting

5.2.1 The Analytics Dashboard

The dashboard presents the structure of the document itself rather than answers to questions. It reports node counts by type, the total number of responsibility entries, the distribution of authority codes, the roles most frequently involved, the proportion of unresolved roles, and the graph totals. It is filterable by chapter, and its charts are linked, so selecting an authority code re-scopes the roles chart to show who most often holds that particular authority.

Two of its indicators describe the extraction rather than the document. The average number of responsibilities per record and the proportion of records with no direct responsibilities are data quality signals that were invisible before the document was structured, and both were used during development to detect regressions, since a sudden change in either after a parser change is a reliable sign that something has broken.

5.2.2 Governance Indicators

The dashboard's original indicators describe the data. The governance indicators described here describe what the data means for the institution, and all of them are computable from the existing knowledge graph without any additional source.

Table 5.1: Governance indicators derived from the knowledge graph.

Indicator	Definition and interpretation
Authority concentration	Share of activities for which exactly one role holds approval authority. A high value identifies single points of failure, where one absence blocks a process.
Approval span per role	Number of activities each role approves. Ranks roles by the breadth of authority they carry and surfaces disproportionate concentration.
Segregation of duties	Share of activities where the initiating role differs from the approving role. A standard internal-control measure, computed here directly from the matrix.
Verification coverage	Share of activities carrying at least one recorded check-and-verify obligation. Measures how much of the Bank's activity has a mandatory second pair of eyes.
Notification breadth	Mean number of informed parties per activity, with distribution. Indicates how widely decisions are communicated.
Threshold banding rate	Share of activities whose approver depends on a monetary band. Quantifies how far delegation is value-sensitive rather than flat.
Authority chain depth	Mean number of distinct roles involved in an activity across all authority types. A proxy for procedural weight.
Documentation completeness	Share of activities with no recorded responsibility of any kind, excluding structural containers. Flags gaps in the matrix itself.
Redirect dependency	Share of activities whose responsibilities are reachable only by following a cross-reference. Measures how much of the document is indirect.

Indicator	Definition and interpretation
Role redundancy	Share of activities with two or more roles able to approve. Read together with authority concentration, it separates resilience from ambiguity.

Two of these deserve comment. Authority concentration and role redundancy are complements, and reading either alone is misleading: an activity with a single approver is efficient and fragile, while one with several is resilient and potentially ambiguous about who is accountable. Presenting both, per chapter, lets a reader see which pattern dominates where. Segregation of duties is the indicator most likely to interest an audit function, because it is a recognised control principle that the matrix encodes implicitly and no one currently measures.

5.2.3 Usage Indicators

The second family describes how the system is used, and becomes available once the authenticated query log described earlier begins accumulating records.

Table 5.2: Usage indicators derived from the query log.

Indicator	Definition and interpretation
Resolution rate	Share of questions resolved to an activity, against those refused or returned for clarification. The headline measure of whether the system is answering its users.
Method mix	Proportion resolved by identifier, by free-text search, by auxiliary handlers, or refused. Shifts in this mix indicate changes in how staff are using the tool.
Mean resolution confidence	Average similarity score for free-text resolutions, with its distribution. A downward trend suggests questions are drifting away from the document's vocabulary.
Grounding fallback rate	Share of generated answers rejected by the verification step. A direct, continuous measurement of model faithfulness in production rather than in testing.
Response latency	Median and 95th percentile, separated by cold and warm start. Relevant given the hosting tier.

Indicator	Definition and interpretation
Query concentration	Most frequently asked activities, processes, and roles. Identifies which parts of the document carry real demand.
Unresolved question backlog	Questions that failed to resolve, retained as a work list. Directly prioritises which chapters to extract next.
Language distribution	Share of questions by detected language. Evidence for whether the multilingual capability is used.
Conversation depth	Mean follow-up questions per conversation. A proxy for whether context carry-over is serving users.
Repeat question rate	Share of questions repeated by the same account within a short window. A proxy for answers that were returned but not understood.

The grounding fallback rate is the one worth highlighting, because it converts a design property into an operational metric. Chapter 4 describes the verification step as the mechanism that makes the language model safe to use. Logging how often it fires turns that claim into something measurable: a rising fallback rate is direct evidence that a model, or a change to the prompt, has become less faithful, and it can be observed without any manual review of answers.

The unresolved question backlog is the most operationally useful. It converts failure into a prioritised work list, since the questions users ask and the system cannot answer are precisely the evidence needed to decide which chapter of the remaining two hundred pages to process next.

5.2.4 Forecasting

Forecasting requires a quantity that varies over time, and the DAM itself does not: it is static reference data. The forecastable quantity in this system is demand, not content, and the analytics layer therefore forecasts consultation activity rather than the document.

5.2.4.1 Rationale

Institutional activity is cyclical. Mission programmes are quarterly, project reviews are annual, and budget and procurement cycles have their own rhythm. If consul-

tation of the authority matrix tracks the activities the matrix governs, then query volume should carry seasonal structure, and a forecast of that structure supports concrete decisions: when to pre-warm the hosted instance, when support demand will peak, and which chapters will be in demand in the coming period.

5.2.4.2 Method

The target series is query count aggregated weekly, both overall and per chapter, drawn from the query log. Modelling proceeds in three steps. Seasonal-trend decomposition separates the series into trend, seasonal, and residual components, which is diagnostic before it is predictive: it establishes whether seasonality is present at all. Holt-Winters exponential smoothing then provides short-horizon forecasts with additive trend and seasonal terms, and a seasonal ARIMA model is fitted as a comparison. All three are available in a single statistical library already compatible with the stack.

Evaluation uses rolling-origin backtesting rather than a single split, reporting mean absolute error and mean absolute percentage error across folds. The comparison that matters is against a seasonal-naïve baseline, which predicts each week from the same week in the previous cycle. A model that cannot beat that baseline does not justify its complexity and is not deployed.

5.2.4.3 The Data Requirement, Stated Plainly

Seasonal structure cannot be estimated from a short series. Reliable weekly seasonality needs on the order of twelve to sixteen weeks of logged activity, and annual cycles need considerably more. Until that history exists, the module reports insufficient data and displays the observed series without a projection.

That behaviour is a deliberate design decision and not a placeholder. A forecasting panel that produces a confident line from three weeks of data would be exactly the failure mode this system was built to avoid: a plausible answer with nothing behind it. The same principle that makes the agent refuse an unknown identifier applies to the analytics layer refusing an unsupportable forecast.

5.2.4.4 Bottleneck Projection

A second analytical capability needs no accumulated history, because it is a ranking rather than a time series. Combining approval span from the graph with query concentration from the log identifies the roles that sit on the most heavily consulted approval paths, and projects which of them are likely to become bottlenecks as consultation grows.

This is presented as a projection under stated assumptions rather than as a prediction, and the assumptions are shown alongside it: that consultation frequency approximates activity frequency, and that the distribution of demand remains broadly stable. Both are plausible and neither is verified, so the output is framed as a prompt for investigation rather than as a finding.

5.2.4.5 What Is Deliberately Not Forecast

Two attractive-sounding capabilities are excluded for the same reason, which is that the data to support them does not exist within this project.

Forecasting actual approval workload, meaning how many approvals a given role will face next quarter, would require the Bank's transaction and project volumes. Those are not part of the DAM and were not available. Forecasting revisions to the matrix itself would require a machine-readable history of previous versions, and only the current version exists in that form. Both are recorded here as extensions that become possible if the corresponding data is supplied, rather than as capabilities the system claims.

5.3 REST Interface

Table 5.3: REST API endpoints exposed by the backend.

Endpoint	Method	Purpose
/api/auth/login	POST	Exchanges credentials for a signed access token. Rate-limited.
/api/auth/me	GET	Returns the authenticated account and its role.
/api/ask	POST	Core endpoint. Accepts a question, model mode, optional carried-over activity, and target language. Returns the answer, the deterministic answer, the resolved identifier, resolution method and score, the backing facts, and language and model metadata.
/api/documents	POST	Attaches a document to a conversation. Validates type and size, extracts text, discards the binary.

Endpoint	Method	Purpose
/api/conversations/{id}/export	GET	Returns the conversation as a PDF including provenance metadata.
/api/conversations/{id}/share	POST	Creates an expiring read-only snapshot and returns its token and URL.
/api/conversations/shared/{token}	GET	Returns a shared snapshot. No account required; the token is the credential.
/api/conversations/{id}/share	DELETE	Revokes an existing share before expiry.
/api/health	GET	Liveness check, used by the platform and the status indicator. Reports nodes loaded.
/api/llm/config	GET	Reports the model names each mode would currently use.
/api/dashboard/summary	GET	Aggregate statistics, per-chapter breakdowns, and graph totals. Administrator only.
/api/analytics/usage	GET	Usage indicators over a requested period. Administrator only.
/api/analytics/forecast	GET	Forecast series with intervals, or an insufficient-data response. Administrator only.

5.4 Testing and Validation

The system carries 374 automated tests across 23 files. The number matters less than their provenance: these are not tests written to reach a coverage figure. Almost every one exists because a specific defect was found, and the test was written to pin the corrected behaviour so it could not silently return.

Table 5.4: Breakdown of the automated test suite.

Test file	Tests	Area covered
test_task_blocks.py	25	Task boundary detection, title attachment, threshold-variant hierarchy
test_row_merge.py	5	Split-row merging in the Non-Sovereign Operations chapter

Test file	Tests	Area covered
test_column_roles.py	8	Rotated header extraction and role attribution
test_title_corruption.py	10	Character-level title recovery for cross-row bleeding
test_metadata.py	11	Reference extraction and footnote-digit stripping
test_glossary_parsing.py	7	Abbreviations list extraction
test_build_nodes.py	7	Node assembly and hierarchy correctness
test_graph.py	7	Graph construction and edge validation
test_nodes_cache.py	2	Lossless cache save and load
test_search.py	8	Identifier fast path and free-text resolution
test_typo_correct.py	18	Typo-tolerance thresholds
test_smalltalk.py	40	Greeting, farewell, and capability detection
test_action_codes.py	20	Authority-code explanation and carry-over exclusion
test_glossary_agent.py	32	Glossary lookup and inline acronym expansion
test_agent.py	29	Answer orchestration, mandatory notes, context carry-over
test_llm.py	15	Provider mocking, mode resolution, failure handling
test_generate.py	19	Verification-step correctness and answer-length scaling
test_translate.py	26	Language detection and translation handling
test_tone.py	25	Tone pre-filter, classification, prefix application
test_backend.py	29	End-to-end API behaviour
test_dashboard_data.py	15	Dashboard metrics and chapter scoping
test_frontend_source.py	16	Structural checks on the web client source

Test file	Tests	Area covered
Total	374	across 23 files

Automated tests were not the only verification. Extraction correctness was checked by comparing specific records against the printed pages they came from, which is the only method that catches a plausible-looking wrong answer, and the running system was exercised through its own HTTP interface with real questions before each significant change was accepted.

Two defects were identified during a late verification pass and are recorded here rather than omitted. On activities 1.116.1 and 1.116.2 a column header was attributed as though it were a role, so a responsibility is reported against a role name that does not exist in the organisation. On the same two records, two adjacent footnote digits were fused into a single number, producing a footnote reference that cannot correspond to any real footnote. Both are localised to those two records, both are visible in a live answer, and both are scheduled for correction alongside the footnote extraction work described in the general conclusion.

5.5 Deployment

The application is containerised, with a multi-stage build that compiles the web client and then packages it alongside the Python service. A companion composition adds a local model container for offline use, and the cloud provider needs no container at all since it is reached over the network in the same way from inside or outside the container.

Deployment targets a free hosting tier, with automatic redeployment on each change to the main branch. The first deployment attempt failed, and the reason is instructive. The service parsed the PDF during startup, which pushed memory use past the platform's limit and caused the process to be terminated before it could serve anything. The fix was to move parsing to build time and ship the serialised node set, so startup became a file load. Startup memory fell below the limit and startup time fell with it.

Conclusion

The delivered platform is a working application rather than a prototype: authenticated access with role separation, a conversational interface accepting typed and spoken questions in five languages, document attachment with an explicit trust boundary, exportable and shareable conversations carrying their own provenance,

an analytics layer covering both governance and usage indicators, and a forecasting capability that declines to forecast until it has the data to do so honestly. Behind it sit 374 automated tests and an automated deployment. The deployment failure described above is included because it illustrates the same pattern as the rest of the project, where an assumption that looked reasonable held until it met the real environment and was corrected once it did.

General Conclusion

This work began with a document that answers an important question badly, not because its content is wrong but because its form makes retrieval expensive. The DAM states who may approve, check, review, initiate, and be informed of each of the Bank's activities, and it states it inside wide tables with sideways headers and a compact notation that resists both manual lookup and automated processing.

What was built in response is a system that separates the question of what the facts are from the question of how to say them. The document was parsed into 327 validated records carrying 1,776 responsibility entries, with under one percent of those entries left unresolved, and turned into a graph of 459 nodes and 2,108 edges. Questions are resolved to activities through an exact identifier path and a lexical search layer, answers are constructed from records in that graph, and a language model rewrites those answers into natural sentences under a verification step that rejects any rewriting which drops or alters a fact. Around that core sits an authenticated platform offering voice input, document attachment with an explicit trust boundary, exportable and shareable conversations, a fully localised interface, and an analytics layer carrying governance indicators, usage indicators, and a forecasting capability.

Three conclusions seem worth drawing from the work.

The first concerns document extraction. Every substantial defect encountered during this project produced output that looked entirely plausible, and none would have been caught by checking that the code ran without error. They were caught by comparing extracted records against the pages they came from. For documents whose structure is visual, that comparison is not a quality-assurance nicety; it is the only method that works.

The second concerns language models in factual settings. A prompt instructing a model not to invent facts is a request, not a guarantee. Making it a property of the system required a mechanical check outside the model, comparing its output against the facts it was given. The check is simple, it is cheap, and it is the reason a generative component could be used at all in a compliance context. The decision to log how often that check fires, described in Chapter 5, extends the same thinking: a safety property that is not measured in production is an assumption rather than a guarantee.

The third concerns refusal. The system's willingness to say that something does not exist, that a question falls outside what it can answer, or that there is not yet

enough data to produce a forecast, is what gives its positive answers meaning. A tool that always produces an answer supplies no information about which of its answers to trust.

Limitations

Three chapters of the DAM were processed out of a document exceeding two hundred and fifty pages. The remaining chapters appear to use the same conventions, but that expectation is untested.

Footnote handling is incomplete, and it is the most substantive gap. Footnote reference numbers are extracted and attached to responsibilities, and 113 of the 1,776 responsibility entries carry them, but the footnote text printed at the foot of each page is not parsed. The system can therefore state that a qualification applies to a responsibility without being able to say what that qualification is. Since footnotes in this document carry conditions such as delegation ceilings and prior-approval requirements, an answer that names an approver while omitting the condition attached to that approval is accurate and potentially misleading at the same time. Two related extraction defects on activities 1.116.1 and 1.116.2, a column header attributed as a role and two footnote digits fused into one number, belong to the same area of the pipeline.

Work on this was deliberately not begun without first settling a question the schema itself flags: whether footnote numbering restarts at each chapter or at each process. Analysis of the extracted references indicates chapter-level scope, since eleven distinct footnote numbers recur across more than one chapter while numbering within a chapter runs as a single sequence rather than restarting at each process. That finding still warrants confirmation against two printed pages before the parser is built on it, which is exactly the discipline the rest of this project followed.

Retrieval is lexical, so a question phrased entirely in vocabulary the document does not use will resolve poorly. And the system reflects one version of one document, with no mechanism for detecting that the underlying DAM has been revised, which in an institutional setting is an operational concern rather than a theoretical one.

Perspectives

The most direct continuation is to complete footnote extraction, which closes the gap described above and removes the two known defects alongside it, and then to process the remaining chapters, which mainly tests whether the parsing rules generalise as expected.

Beyond that, a hybrid retrieval layer combining the current lexical matching with a semantic component would improve resolution for questions phrased in outside vocabulary, provided the verification step remains in place unchanged. A versioning mechanism able to compare a newly issued DAM against the current knowledge base and report what changed would turn a static reference into something maintainable.

The analytics direction is the one with the most room. The governance indicators described in Chapter 5 measure properties of the matrix that no one currently measures, and the forecasting capability becomes meaningful as soon as the query log has accumulated enough history. Extending the same graph analysis toward questions the printed document cannot easily answer, such as where authority concentrates, which roles appear in an unusually large number of approval paths, and where a single point of failure exists in a chain of obligations, would make the system a governance instrument rather than only a lookup tool.

Bibliography and Webography

Institutional Sources

- [1] African Development Bank Group. *Delegation of Authority Matrix*. Internal reference document, PDF, more than 250 pages. Source data for this work; an internal Bank document, not publicly published.
- [2] African Development Bank Group. Institutional website. <https://www.afdb.org>

Methodology

- [3] Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., and Wirth, R. *CRISP-DM 1.0: Step-by-step Data Mining Guide*. SPSS Inc., 2000.
- [4] Wirth, R. and Hipp, J. "CRISP-DM: Towards a Standard Process Model for Data Mining." *Proceedings of the Fourth International Conference on the Practical Application of Knowledge Discovery and Data Mining*, 2000.

Information Retrieval, Knowledge Graphs, and Language Models

- [5] Manning, C. D., Raghavan, P., and Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [6] Hogan, A. et al. "Knowledge Graphs." *ACM Computing Surveys*, vol. 54, no. 4, 2021.
- [7] Lewis, P. et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems*, 2020.
- [8] Ji, Z. et al. "Survey of Hallucination in Natural Language Generation." *ACM Computing Surveys*, vol. 55, no. 12, 2023.

Time Series and Forecasting

- [9] Hyndman, R. J. and Athanasopoulos, G. *Forecasting: Principles and Practice*, 3rd edition. OTexts, 2021. <https://otexts.com/fpp3/>

[10] Cleveland, R. B., Cleveland, W. S., McRae, J. E., and Terpenning, I. "STL: A Seasonal-Trend Decomposition Procedure Based on Loess." *Journal of Official Statistics*, vol. 6, no. 1, 1990.

Security and Web Standards

[11] Jones, M., Bradley, J., and Sakimura, N. *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force, 2015. <https://datatracker.ietf.org/doc/html/rfc7519>

[12] Provos, N. and Mazières, D. "A Future-Adaptable Password Scheme." *Proceedings of the USENIX Annual Technical Conference*, 1999.

[13] World Wide Web Consortium. *Web Speech API Specification*. <https://wicg.github.io/speech-api/>

[14] OWASP Foundation. *OWASP Top Ten*. <https://owasp.org/www-project-top-ten/>

Technical Documentation

[15] pdfplumber documentation. <https://github.com/jsvine/pdfplumber>

[16] NetworkX documentation. <https://networkx.org/documentation/stable/>

[17] scikit-learn documentation, feature extraction from text. https://scikit-learn.org/stable/modules/feature_extraction.html

[18] statsmodels documentation, time series analysis. <https://www.statsmodels.org/stable/tsa.html>

[19] FastAPI documentation. <https://fastapi.tiangolo.com/>

[20] Pydantic documentation. <https://docs.pydantic.dev/>

[21] React documentation. <https://react.dev/>

[22] ReportLab documentation. <https://docs.reportlab.com/>

[23] Docker documentation. <https://docs.docker.com/>

[24] Render deployment documentation. <https://render.com/docs>

[25] Groq API documentation. <https://console.groq.com/docs>

[26] Ollama documentation. <https://github.com/ollama/ollama>